



UNIVERSITY OF CAPE TOWN

TIME SERIES
PORTFOLIO

The Temporal Shift

Author:
Isabella Lethbridge

Student Number:
LTHISA001

October 5, 2026

Table of contents

Overview	4
Overview	4
Portfolio Structure	4
Reflective Commentary	4
Learning Achievements	4
Methodological Challenges	4
Ethical and Reproducibility Considerations	4
Future Areas for Development	4
AI Use Log	4
Section 1	5
Foundations	5
Learning outcomes	5
What is a time series?	5
Qualitative, Continuous and Discrete Series	6
Temporal granularity and the aggregation effect	8
One realisation of a stochastic process	8
The Forecasting Problem	8
What is forecasting?	8
Exploring a Series	9
The classical decomposition	9
Two ways of looking at a series	9
The components of a series	9
Time Series Analysis in R	11
AirPassengers dataset	11
Handling dates and aggregation	11
Helper conversions	12
Subsetting with <code>window()</code>	12
Lags and Differences	12
Combing series	13
Missing data	13
Time series plots	14
Check attributes of time series	15
Benchmarks and Forecast Accuracy	15
The benchmarks applied	15
Cross-validation	16
Strategies	16

Manual cross-validation	17
Cross-validation using <code>tsCV()</code>	18
Exercises	18
Operators, Stationarity and White Noise	19
Lag and lead operators	19
Lag and lead in R	19
Differencing	20
Weak and strict stationarity	20
White noise	23
Linear Models for the Conditional Mean	23
The $MA(q)$ model	23
The $AR(p)$ model	23
The $ARMA(q, q)$ model	23
The $ARIMA(p, d, q)$ model	25
The $SARIMA(p, d, q)(P, D, Q)_m$ model	25
Conditional Variance Models	29
Problem 1	29
Spectral Analysis	32
Section 2	32
Data Statement	32
Teaching Session Summaries	32
Worked Tutorials & Software Implementations	32
Topic Exploration	32
Projects	32
Overview	32
Project 1	33
Key Findings	33
Reflections	33
Links	33
Project 2	36
Key Findings	37
Reflections	37
Links	37
Predicting drought in Southern Africa	37
Exploratory Data Analysis	38
Model fitting	47
Model fitting	56
Dam levels	56
Sea surface temperature	56

Project 3	56
Key Findings	56
Reflections	61
Links	61
Project 4	61
Key Findings	61
Reflections	61
Links	61
Journal	61
Appendix	61
R Code & Scripts	61
Data Access & Repositories	61
Computational Notebooks	62

Overview

Overview

Welcome to my portfolio for Time Series Analysis. This document showcases my learning progression, statistical understanding, computational applications, and reflections across the course.

Portfolio Structure

- [Reflective Commentary](#)
- [AI Use Log](#)
- [Section 1](#)
- [Section 2](#)
- [Projects](#)
- [Journal](#)
- [Appendix](#)

Reflective Commentary

Learning Achievements

[cite: 1]

Methodological Challenges

[cite: 1]

Ethical and Reproducibility Considerations

[cite: 1]

Future Areas for Development

[cite: 1]

AI Use Log

Date	Tool Used	Prompt / Task Description	Link to Conversation	Verification & Changes Made
YYYY-MM-DD	ChatGPT / Claude	Debugging R code for model fit	Link	Checked output against manual calculations[cite: 1]

Section 1

Foundations

Learning outcomes

1. Explain what a time series is and how it differs from cross-sectional data, and say why temporal dependence calls for its own methods.
2. Treat an observed series as one realisation of a stochastic process.
3. Explain what forecasting is, judge when a process is predictable, and keep forecasts, goals and plans separate.
4. Match a forecast horizon to the decision it supports, and read a forecast as a random variable rather than a single number.
5. Identify trend, seasonality, cycle and irregular variation in a plot, and describe the shapes each of them takes in practice.
6. Produce the four benchmark forecasts, and evaluate any competing model against them on data it has not seen.
7. Use the lag and lead operators, and the algebra built on them, to write models compactly.
8. Define weak stationarity, say which of its conditions a given series violates, and choose a transformation that restores it.
9. Define white noise, MA, AR, ARMA, ARIMA and SARIMA processes, derive their basic properties, and identify them from an ACF and a PACF.
10. Fit, diagnose and forecast these models in R, following the workflow of Box and Jenkins, and test residuals against the white noise standard.
11. Explain volatility clustering and fit ARCH and GARCH models to it

What is a time series?

Time series is a single measurement taken at multiple points across time. Technically we are modelling the same thing over and over again (this makes sense as to why there is sequential correlation).

i Note

Time series model dependence through sequential observations.

! Assumption

Equally spaced measurements.

! Assumption

Autocorrelation across time.

Qualitative, Continuous and Discrete Series

Example: Continuous series

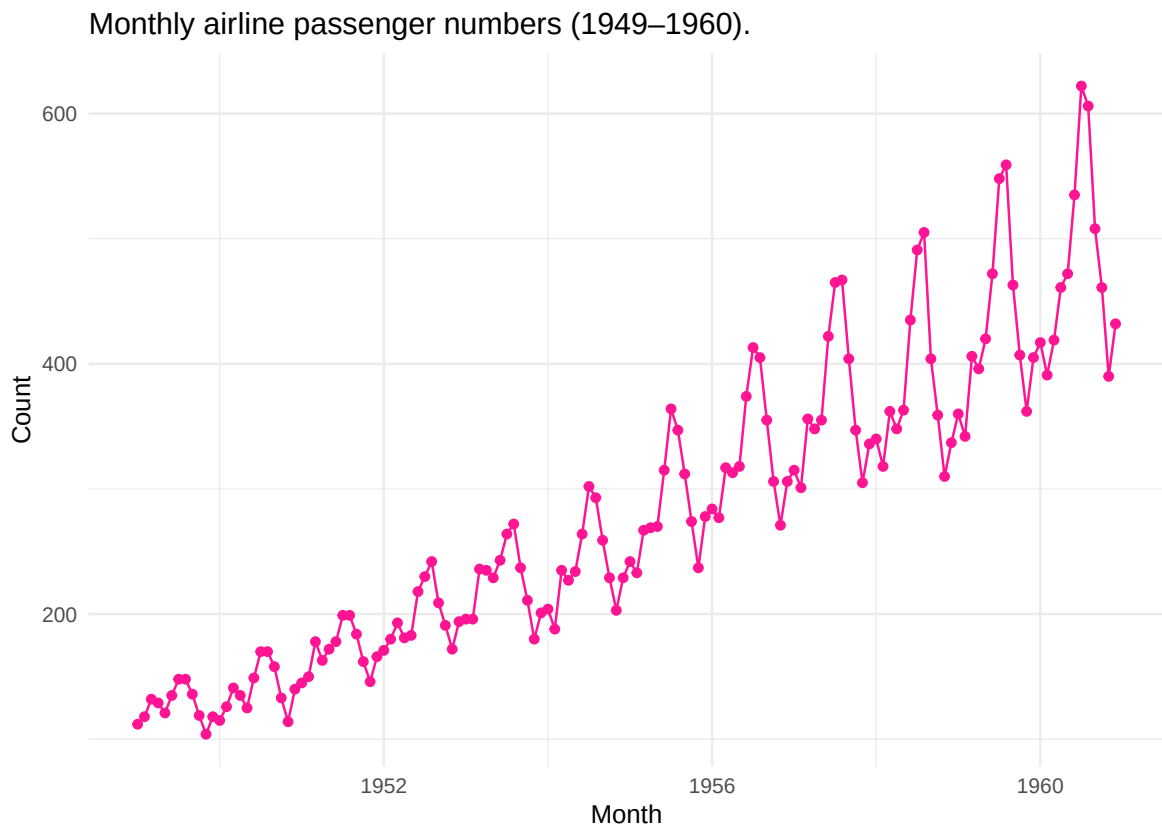


Figure 1: Time series plot of the number of monthly airline passengers from 1949 to 1960.

Figure 1 plots the number of monthly airline passengers over a 12 year period. This figure clearly illustrates the concepts of seasonality and trend.

Example: Discrete series

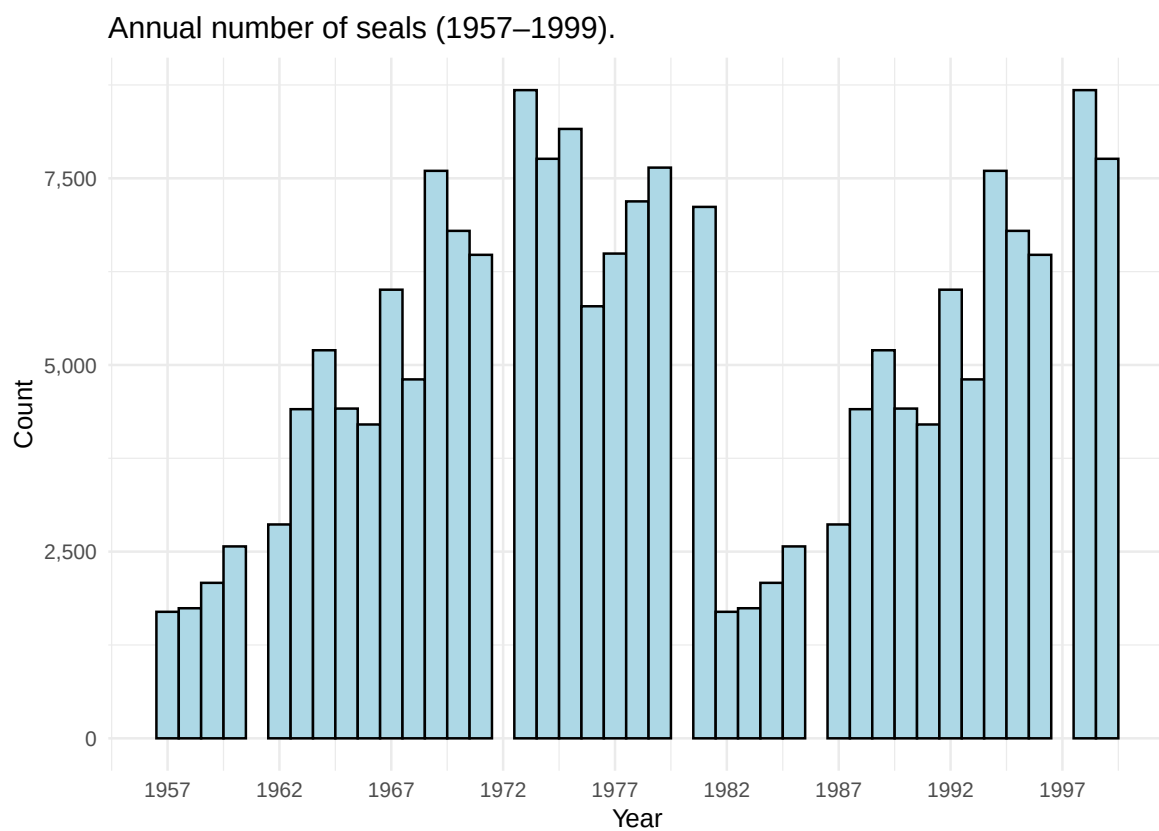


Figure 2: Time series plot of the number of seals from 1975 to 1999.

Figure 2 plots the changing number of observed seals over a period of 25 years.

What to do when you have missing observations in a time series?

In discrete time series to deal with missing observations you can express the model in state-space form (ARIMA or Structural Time Series). For a missing observation y_t , skip the update step in the kalman filter and only run the prediction step. The likelihood is evaluated only over the observed time points.

Kalman filtering

Kalman filtering (linear quadratic estimation) is an algorithm that uses a series of measurements over time to estimate unknown variables, by estimating a joint probability distribution over the variables for each time-step.

In continuous time series, missing observations look like irregularly spaced sampling intervals. You can model the underlying system using Stochastic Differential Equations (CARMA) or you can aggregate the irregular time steps into regular, discrete intervals.

Temporal granularity and the aggregation effect

Aggregating up can either smooth out high-frequency structure (stock returns, etc) or can reveal low-frequency structure (Sales, etc).

One realisation of a stochastic process

Noise in the data introduces different realisations. If your model fits one dataset perfectly then it is overfitting.

The Forecasting Problem

What is forecasting?

Forecasting is a prediction about what will happen in the future based on evidence that we have (with some level of uncertainty). This uncertainty gives rise to forecast intervals (not just point estimates). Goal setting is what we want to happen in the future. Planning helps us reach our goal.

Note

A series can only be called predictable if the interval and horizon are stated.

Forecast → Goal → Plan → Implementation → Outcome → Forecast ...

A forecast horizon is how far into the future you try and predict values. When looking at time series data we are looking for stable, informative patterns (not just a lot of data).

A random walk is a process where memory is permanent. Every future shock accumulates endlessly resulting in forecast variance growing linearly. A stationary AR is a process where memory decays geometrically. Shocks dies out, forcing the forecast back to the process mean.

Stationary AR(1) process, h step forecast error variance:

$$\text{Var}(e_{T+h}) = \sigma^2 \frac{1 - \phi^{2h}}{1 - \phi^2} \rightarrow \frac{\sigma^2}{1 - \phi^2} \quad \text{as } h \rightarrow \infty$$

i Note

Forecasts are random variables.

Exploring a Series

The classical decomposition

Additive: $y_t = T_t + S_t + C_t + I_t$

Multiplicative: $y_t = T_t \cdot S_t \cdot C_t \cdot I_t$

The nature of the seasonality in a series (additive, multiplicative or drifting) determines whether log transformation or differencing are required.

Two ways of looking at a series

1. Time domain - this looks at previous observations

You would use time domain when estimating predictive conditional expectations (forecasting at time $t + 1$ given $t, t - 1$), when dealing with short sample sizes.

A lag is the separation between two observations measured in time units.

i Note

Plotting a series against its own first lag is the simplest possible diagnostic for dependence.

2. Frequency domain - this looks at previous trends

You would use frequency domain when identifying hidden cycles/patterns or when analysing long-memory dynamics.

The components of a series

Trend is the long run direction of movement. Seasonality is th pattern that repeats at a fixed and known interval. Cycles are long oscillations with no fixed length. Irregular variation (noise) is random variation unexplained by the former.

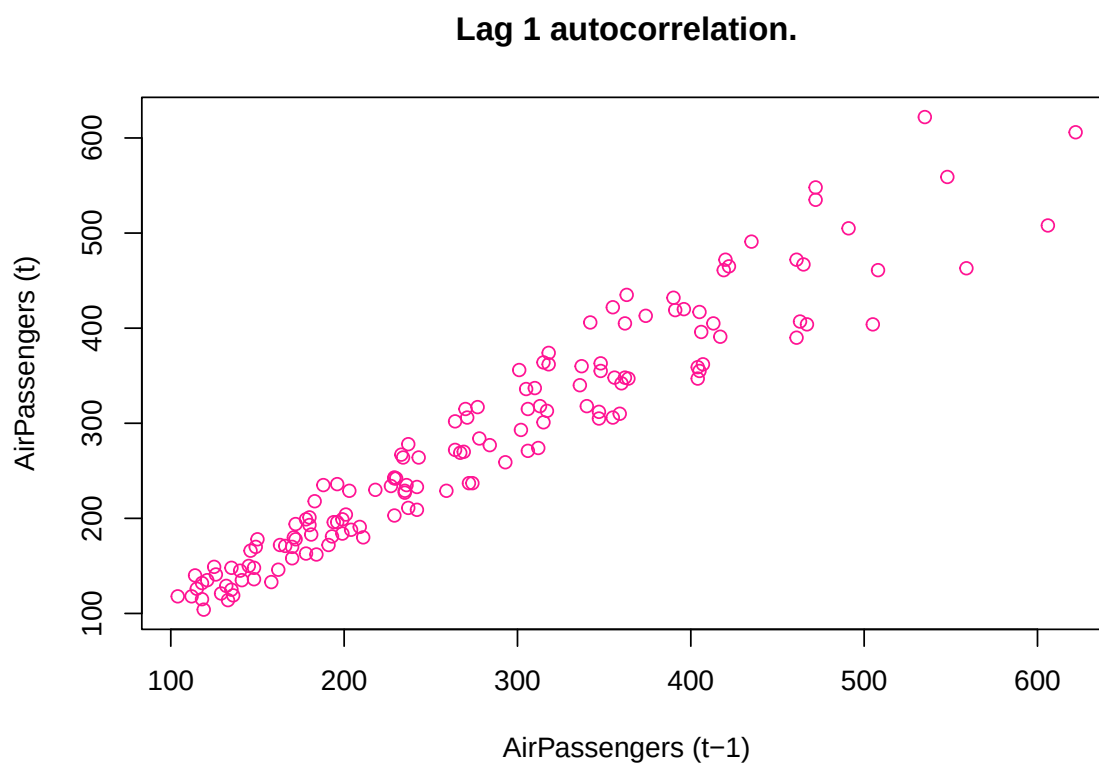


Figure 3: Simple diagnostic test for dependence. Autocorrelation of a series against its own first lag.

Time Series Analysis in R

AirPassengers dataset

```
1 # Run once - install anything missing
2 pkgs <- c("tidyverse", "tsibble", "feasts", "fable", "timetk",
3           "modeltime", "kableExtra", "here")
4 install.packages(pkgs[!pkgs %in% installed.packages()[, "Package"]])
```

Handling dates and aggregation

AirPassengers is already a time series object but to convert a dataset to a time series object you use: `ts(c(), start = c(year, month), frequency = #obs/unit time)`

```
1 # Read in data:
2 data(AirPassengers)
3 # View(AirPassengers)
4
5 print(AirPassengers)
```

	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
1949	112	118	132	129	121	135	148	148	136	119	104	118
1950	115	126	141	135	125	149	170	170	158	133	114	140
1951	145	150	178	163	172	178	199	199	184	162	146	166
1952	171	180	193	181	183	218	230	242	209	191	172	194
1953	196	196	236	235	229	243	264	272	237	211	180	201
1954	204	188	235	227	234	264	302	293	259	229	203	229
1955	242	233	267	269	270	315	364	347	312	274	237	278
1956	284	277	317	313	318	374	413	405	355	306	271	306
1957	315	301	356	348	355	422	465	467	404	347	305	336
1958	340	318	362	348	363	435	491	505	404	359	310	337
1959	360	342	406	396	420	472	548	559	463	407	362	405
1960	417	391	419	461	472	535	622	606	508	461	390	432

```
1 # Sum by year:
2 annual_sum <- aggregate(AirPassengers, FUN = sum)
3
4 # Mean by year:
5 annual_mean <- aggregate(AirPassengers, FUN = mean)
6
7 print(annual_sum)
```

```
1 Time Series:
2 Start = 1949
3 End = 1960
4 Frequency = 1
5 [1] 1520 1676 2042 2364 2700 2867 3408 3939 4421 4572 5140 5714
```

```
1 print(annual_mean)
```

```
1 Time Series:
2 Start = 1949
3 End = 1960
4 Frequency = 1
```

```
5 [1] 126.6667 139.6667 170.1667 197.0000 225.0000 238.9167 284.0000
   328.2500
6 [9] 368.4167 381.0000 428.3333 476.1667
```

Helper conversions

```
1 t <- time(AirPassengers)
2
3 # Extract Year and Month
4 years <- floor(t)
5 months <- round(12 * (t - years)) + 1
6
7 head(data.frame(Decimal = as.numeric(t), Year = years, Month = months))
```

```
1      Decimal Year Month
2 1 1949.000 1949      1
3 2 1949.083 1949      2
4 3 1949.167 1949      3
5 4 1949.250 1949      4
6 5 1949.333 1949      5
7 6 1949.417 1949      6
```

Subsetting with window()

```
1 # Extract only the second half of 2023
2 sub_ts <- window(AirPassengers, start = c(1950, 7), end = c(1955, 10))
3 print(sub_ts)
```

```
1      Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
2 1950      170 170 158 133 114 140
3 1951 145 150 178 163 172 178 199 199 184 162 146 166
4 1952 171 180 193 181 183 218 230 242 209 191 172 194
5 1953 196 196 236 235 229 243 264 272 237 211 180 201
6 1954 204 188 235 227 234 264 302 293 259 229 203 229
7 1955 242 233 267 269 270 315 364 347 312 274
```

Lags and Differences

```
1 # Shift the time index forward by 1 period (does not move the values)
2 lagged_ts <- stats::lag(AirPassengers, k = -1)
3
4 # First-order differencing (removes linear trend)
5 diff_ts <- diff(AirPassengers, differences = 1)
6
7 # Seasonal differencing (removes a 12-month seasonal pattern)
8 seasonal_diff_ts <- diff(AirPassengers, lag = 12)
9
10 diff_ts
```

```
1      Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
2 1949      6 14 -3 -8 14 13 0 -12 -17 -15 14
3 1950 -3 11 15 -6 -10 24 21 0 -12 -25 -19 26
```

4	1951	5	5	28	-15	9	6	21	0	-15	-22	-16	20
5	1952	5	9	13	-12	2	35	12	12	-33	-18	-19	22
6	1953	2	0	40	-1	-6	14	21	8	-35	-26	-31	21
7	1954	3	-16	47	-8	7	30	38	-9	-34	-30	-26	26
8	1955	13	-9	34	2	1	45	49	-17	-35	-38	-37	41
9	1956	6	-7	40	-4	5	56	39	-8	-50	-49	-35	35
10	1957	9	-14	55	-8	7	67	43	2	-63	-57	-42	31
11	1958	4	-22	44	-14	15	72	56	14	-101	-45	-49	27
12	1959	23	-18	64	-10	24	52	76	11	-96	-56	-45	43
13	1960	12	-26	28	42	11	63	87	-16	-98	-47	-71	42

Figure 3 illustrates the autocorrelation between `AirPassengers` and its lag 1.

Combing series

```

1 ts_a <- window(AirPassengers, start = c(1950, 1), end = c(1955, 12))
2 ts_b <- window(AirPassengers, start = c(1955, 1), end = c(1956, 12))
3
4 # Union: keeps full range, fills gaps with NA
5 combined_union <- ts.union(ts_a, ts_b)
6
7 # Intersect: keeps only the overlapping months
8 combined_intersect <- ts.intersect(ts_a, ts_b)
9
10 combined_intersect

```

```

1      ts_a ts_b
2 Jan 1955 242 242
3 Feb 1955 233 233
4 Mar 1955 267 267
5 Apr 1955 269 269
6 May 1955 270 270
7 Jun 1955 315 315
8 Jul 1955 364 364
9 Aug 1955 347 347
10 Sep 1955 312 312
11 Oct 1955 274 274
12 Nov 1955 237 237
13 Dec 1955 278 278

```

Missing data

```

1 has_na <- anyNA(AirPassengers)
2 cat("Contains missing values:", has_na, "\n")

1 Contains missing values: FALSE

1 # Simple linear interpolation for isolated gaps (base R, no extra
  packages)
2 ts_with_gap <- AirPassengers
3 ts_with_gap[5] <- NA
4 ts_filled <- ts(approx(time(ts_with_gap),
5                      ts_with_gap,

```

```

6         xout = time(ts_with_gap))$y,
7         start = start(ts_with_gap),
8         frequency = frequency(ts_with_gap))
9 ts_filled

```

	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
1949	112	118	132	129	132	135	148	148	136	119	104	118
1950	115	126	141	135	125	149	170	170	158	133	114	140
1951	145	150	178	163	172	178	199	199	184	162	146	166
1952	171	180	193	181	183	218	230	242	209	191	172	194
1953	196	196	236	235	229	243	264	272	237	211	180	201
1954	204	188	235	227	234	264	302	293	259	229	203	229
1955	242	233	267	269	270	315	364	347	312	274	237	278
1956	284	277	317	313	318	374	413	405	355	306	271	306
1957	315	301	356	348	355	422	465	467	404	347	305	336
1958	340	318	362	348	363	435	491	505	404	359	310	337
1959	360	342	406	396	420	472	548	559	463	407	362	405
1960	417	391	419	461	472	535	622	606	508	461	390	432

Time series plots

Figure 1 plots the `AirPassengers` time series.

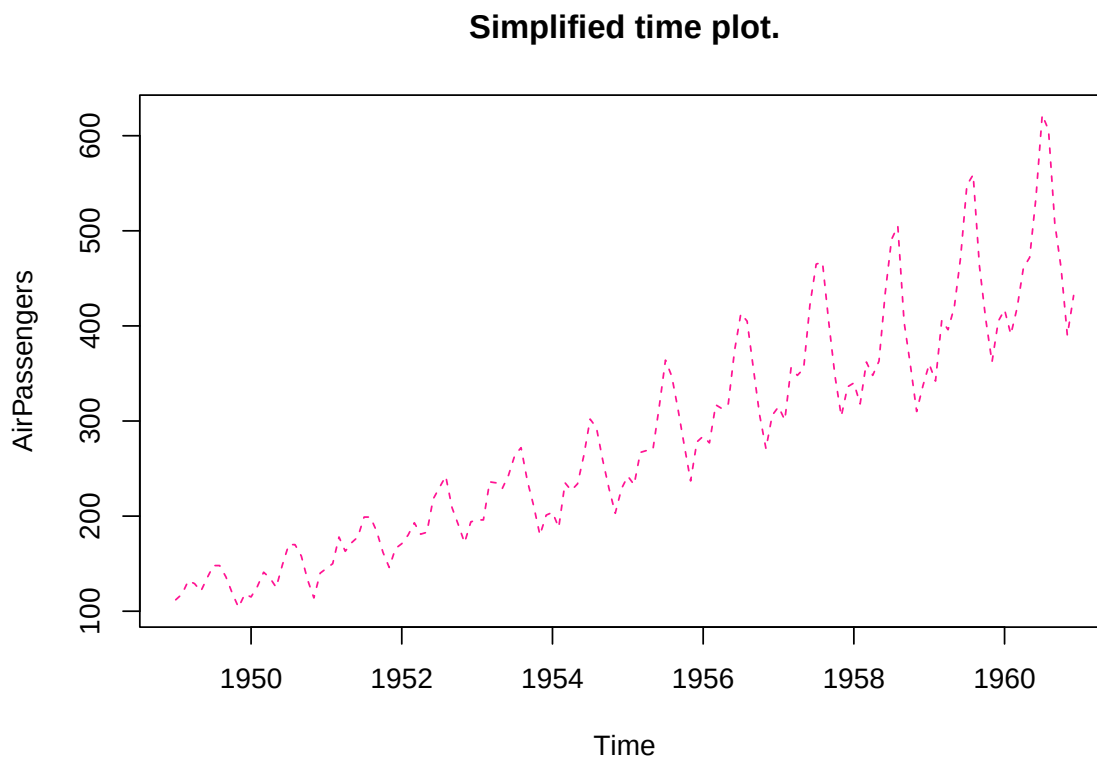


Figure 4: Time series plot of the number of monthly airline passengers from 1949 to 1960.

Check attributes of time series

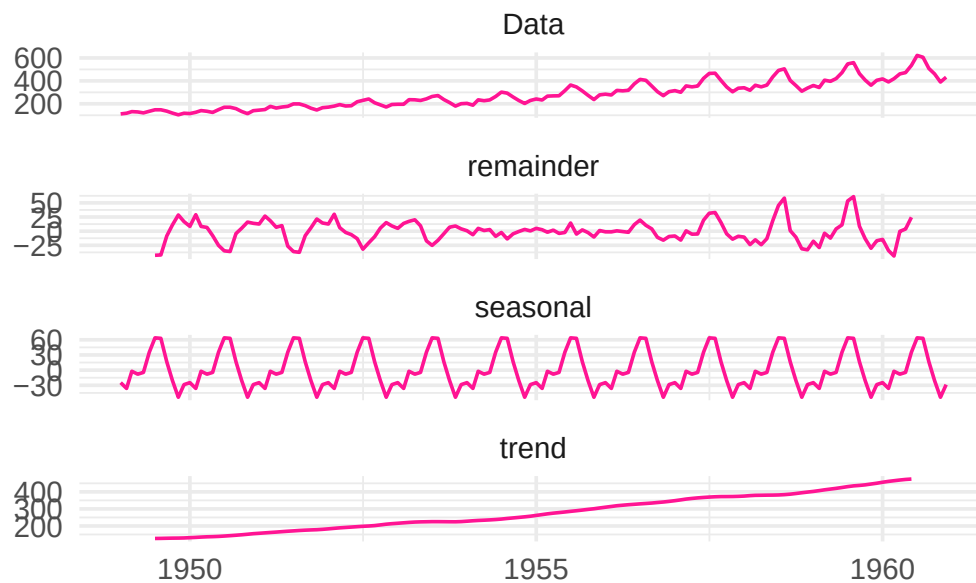
```
1 # Check the attributes
2 cat("Start:", start(AirPassengers), "\n")

1 Start: 1949 1

1 cat("Frequency:", frequency(AirPassengers), "\n")

1 Frequency: 12

1 library(ggfortify)
2 # A first look at structure: naive decomposition (covered in depth in
  Part B)
3 decomp <- decompose(AirPassengers)
4 autoplot(decomp,
5   ts.colour='deeppink')+
6   theme_minimal(base_size = 14)
```



Benchmarks and Forecast Accuracy

The benchmarks applied

Mean - the average of all previous observations is carried into the future.

$$\hat{y}_{T+h|T} = \bar{y}$$

Naive - the last observed value becomes the value that we take into the future.

$$\hat{y}_{T+h|T} = y_T$$

Seasonal naive - the last observed seasonal pattern is carried into the future.

$$\hat{y}_{T+h|T} = y_{T+h-m(k+1)}$$

Drift - The trend of the previous observations is carried into the future.

$$\hat{y}_{T+h|T} = y_T + \frac{h(y_T - y_1)}{T - 1}$$

i Note

Residuals are for diagnostics (validation). Forecast errors are for evaluation (test).

Cross-validation

In time series successive observations are correlated and their ordering carries information about the data generating process. When performing model validation you cannot have future observations predicting the past.

Temporal leakage (look-ahead-bias) is when future observations enter model estimation, this artificially inflates accuracy.

The forecast origin is the last observation available when the forecast is made.

i Note

The test set should be at least as long as the largest horizon of interest.

Strategies

1. Walk-forward validation

The forecast origin advances one step at a time.

2. Expanding window validation

Every historical observation is kept. The training set grows with all past data. When older data is more informative this method reduces estimation variance.

3. Sliding window validation

The training window has a fixed length. The number of training observations stays the same but the forecast origin is moved forward through time. When older data is less informative this method reduces bias.

What is the difference between interpolation and forecasting?

Note

Don't ignore structural breaks when choosing a validation strategy.

Manual cross-validation

Cross-validation is performed using an expanding window approach.

```
1 # Load data:
2 data(AirPassengers)
3 dat = AirPassengers
4
5 # Load libraries:
6 library(forecast)
7
8 n = length(dat) # no. observations
9 train_n = 60    # no. training data points
10 h = 12         # forecast horizon
11
12 # Initialise error matrix:
13 names = c('Mean', 'Naive', 'Seasonal naive', 'Drift')
14 splits = n-train_n-h+1
15
16 res = matrix(NA,
17             nrow=(n-h)-train_n+1,
18             ncol=4,
19             dimnames=list(NULL, names))
20
21 for (i in train_n:(n-h)) {
22   idx = i-train_n+1
23   # Subset training data:
24   train = window(dat,
25                 end=c(1949+(i-1)%/%12, (i-1)%%12+1))
26
27   # Extract test data:
28   test = window(dat,
29                 start=c(1949+i%/%12, i%12+1),
30                 end=c(1949+(i+h-1)%/%12, (i+h-1)%%12+1))
31
32   # Fit model:
33   fits = list(
34     Mean = meanf(train, h=h),
35     Naive = naive(train, h=h),
36     SNaive = snaive(train, h=h),
37     Drift = rwf(train, h=h, drift=TRUE)
38   )
39
40   # Extract rmse for each model:
41   rmse = sapply(fits, \(f) sqrt(mean((as.numeric(test)-as.numeric(f$mean))^2)))
42 }
```

```

43 # Store results:
44 res[idx,] = rmse
45 }
46
47 colMeans(res)

```

	Mean	Naive	Seasonal naive	Drift
	152.87587	74.08791	39.92124	71.96374

Cross-validation using tsCV()

```

1 tsCV_m = tsCV(AirPassengers,meanf,h=1,window=60)
2 m_err = sqrt(mean(tsCV_m^2, na.rm = TRUE))
3
4 tsCV_n = tsCV(AirPassengers,naive,h=1,window=60)
5 n_err = sqrt(mean(tsCV_n^2, na.rm = TRUE))
6
7 tsCV_sn = tsCV(AirPassengers,snaive,h=1,window=60)
8 sn_err = sqrt(mean(tsCV_sn^2, na.rm = TRUE))
9
10 tsCV_d = tsCV(AirPassengers,rwf,h=1,window=60)
11 d_err = sqrt(mean(tsCV_d^2, na.rm = TRUE))
12
13 err_df = cbind(m_err,n_err,sn_err,d_err)
14 colnames(err_df) = c('Mean','Naive','Seasonal naive','Drift')
15 err_df

```

	Mean	Naive	Seasonal naive	Drift
[1,]	102.0794	41.52524	40.61316	41.52524

When comparing the expanding window vs the sliding window, the sliding window gives lower errors for all models except the seasonal naive model. This is because the `AirPassengers` dataset does not have constant variance, as seen in Figure 1.

Exercises

1. Why is ordinary random k-fold cross-validation inappropriate for forecasting?

If you exclude an observation at time $t-1$ but use observation t , you are technically predicting the past based on future observations. Future observations must never be used when estimating a forecasting model as this violates a fundamental principal of forecasting -

2. Define temporal leakage.

Future observations enter model estimation. Reported accuracy looks excellent but is unattainable in practice - the future is unknown when real forecasts are made.

3. Contrast Walk_Forward, Expanding Window and Sliding Window validation.

In expanding window validation every historical observation is kept. The training set grows with all past data. In sliding window validation the training window has a fixed

length. The number of training observations stays the same but the forecast origin is moved forward through time. Walk-forward validation is when the forecast origin advances one step at a time.

4. When would Sliding Window outperform Expanding Window?

When data is non-stationary and older data is less informative.

5. A firm refits its model daily using only the most recent 250 observations. Which strategy is this?

Sliding window

6. Why must temporal ordering be preserved during validation?

Otherwise the model knows things it could not have known at forecasting time. This leads to inflated accuracy that is unattainable in practise.

Operators, Stationarity and White Noise

Lag and lead operators

Lag operator $L_{y_t} = y_{t-1}$ at time t uses the observation recorded at time $t - 1$.

Note

The subscript records where a value came from, not where it is displayed.

Lead operator $F_{y_t} = y_{t+1}$ at time t uses the observation recorded at time $t + 1$, $F = L^{-1}$.

Lag and lead in R

```
1 y <- c(3, 7, 4, 8, 5)
2 c(NA, y[1:(length(y)-1)]) # lag by hand

1 [1] NA 3 7 4 8

1 dplyr::lag(y, n = 1) # same shift

1 [1] NA 3 7 4 8

1 dplyr::lead(y, n = 1) # lead

1 [1] 7 4 8 5 NA

1 lag_twice = dplyr::lag(dplyr::lag(y,n=1),n=1)
2 lag_twice

1 [1] NA NA 3 7 4

1 dplyr::lag(y,n=2)

1 [1] NA NA 3 7 4
```

 Built in `lag()` functions

Don't get confused between `stats::lag()` and `dplyr::lag()`. They do different things!

Differencing

First difference:

$$\Delta y_t = y_t - y_{t-1} = (1 - L)y_t,$$

Seasonal difference (at lag 12):

$$\Delta_{12}y_t = y_t - y_{t-12} = (1 - L^{12})y_t,$$

```
1 # First differences, by hand and with diff().
2 y2 <- c(10, 6, 9, 12, 7)
3 y2 - dplyr::lag(y2, 1)
```

```
1 [1] NA -4  3  3 -5
```

```
1 diff(y2) # Note: no NA added
```

```
1 [1] -4  3  3 -5
```

Weak and strict stationarity

When we have a multiplicative model, with expanding variance, we can log-transform it to obtain an additive model. This removes the variance of seasonality but as seen below the series still has a trend.

Taking differences between observations removes the linear trend and taking seasonal differences removes the seasonal trend.

 Assumption

No trends present in data.

We can remove both the linear and seasonal trend in the data:

```
1 # Check what sasonal and what regular differences are required:
2 nsdiffs(log_AP)
```

```
1 [1] 1
```

```
1 nsdiffs(log_AP)
```

```
1 [1] 1
```

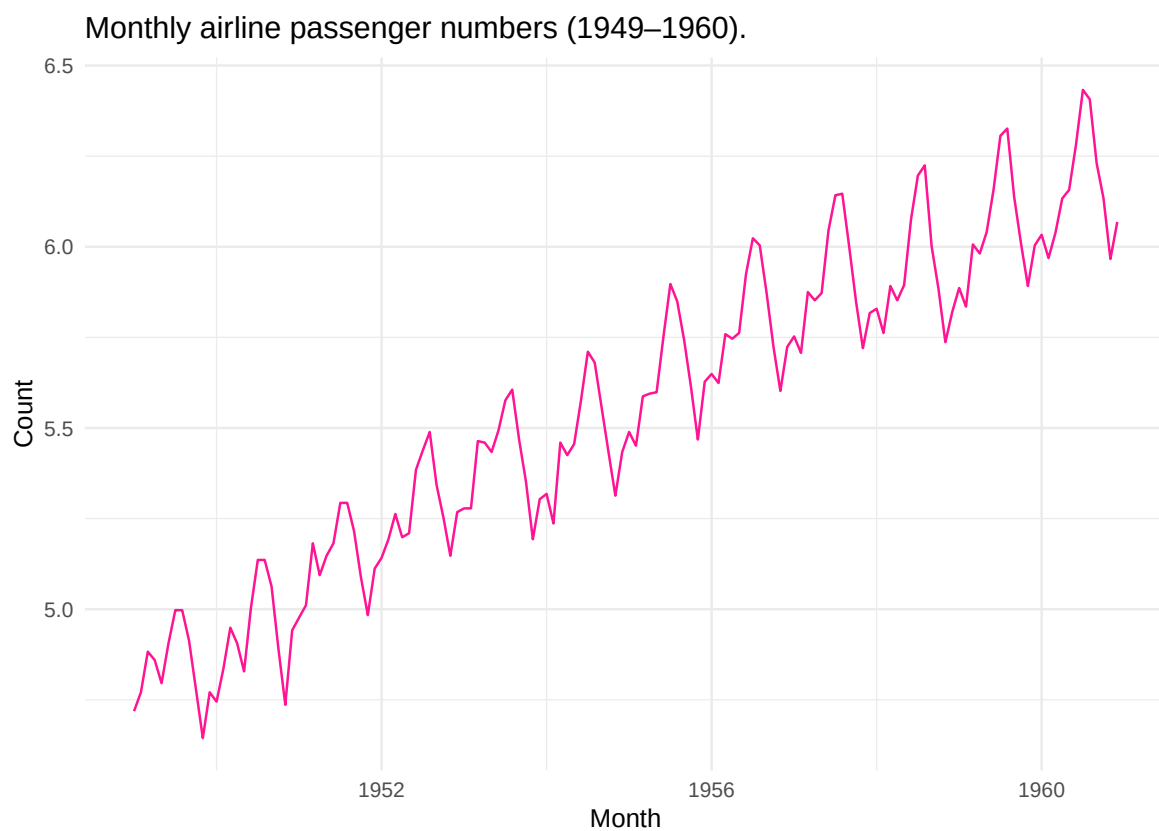


Figure 5: Time series plot of the log-transformed number of monthly airline passengers from 1949 to 1960.

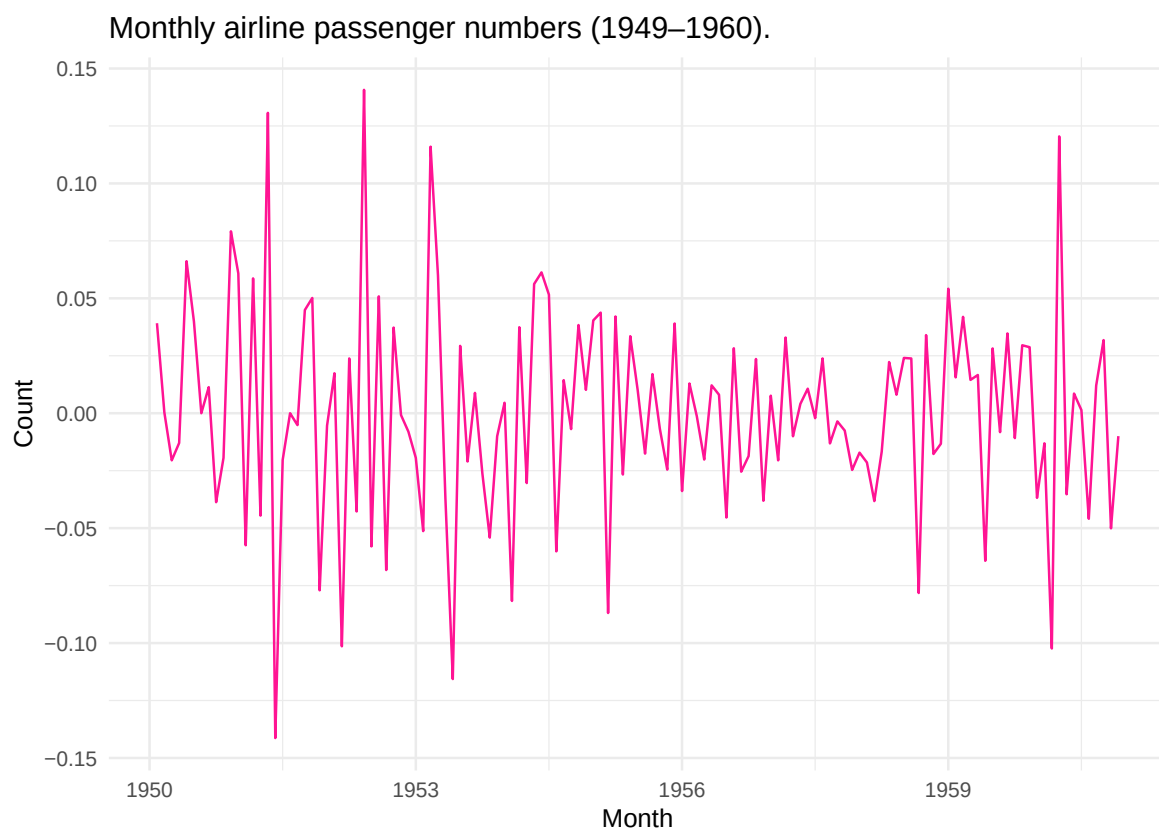


Figure 6: Time series plot of the number of monthly airline passengers from 1949 to 1960 in a stationary distribution.

White noise

Weak white noise only requires uncorrelated observations. Strict white noise requires *iid* observations.

White noise means unpredictable variance, whatever the variance.

Note

Indicator variables can be included to remove trends when the series has structural breaks in it.

Linear Models for the Conditional Mean

Although white noise is uncorrelated, a weighted combination of neighbouring white noise is not.

Filtering is the process of building a new series as a weighted combination of the terms of another.

The MA(q) model

$$y_t = \mu + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \cdots + \theta_q \varepsilon_{t-q}, \quad \varepsilon_t \sim WN(0, \sigma^2).$$

The moving average model is driven by lagged shocks not lagged observations. A MA(q) process is stationary for any values of the coefficients, since it is a finite weighted sum of white noise terms with fixed weights. The coefficients must satisfy invertibility.

The AR(p) model

$$y_t = c + \phi_1 y_{t-1} + \cdots + \phi_p y_{t-p} + \varepsilon_t, \quad \varepsilon_t \sim WN(0, \sigma^2).$$

For the autoregressive model there is a set of correlation in the structure. The current value of a variable depends linearly on its own past. An AR(p) process models memory.

If you are observing spikes outside of the first few this indicates seasonality in the series. Figure 8 suggests that there is still seasonality present in the transformed and differenced `AirPassengers` dataset, however this could also just be because differencing reduced the number of observations resulting in a more noisy dataset.

The ARMA(q, q) model

The ARMA model allows for process that show persistence and short lived shocks simultaneously. The series must be stationary.

$$y_t = c + \sum_{i=1}^p \phi_i y_{t-i} + \varepsilon_t + \sum_{j=1}^q \theta_j \varepsilon_{t-j}, \quad \varepsilon_t \sim WN(0, \sigma^2).$$

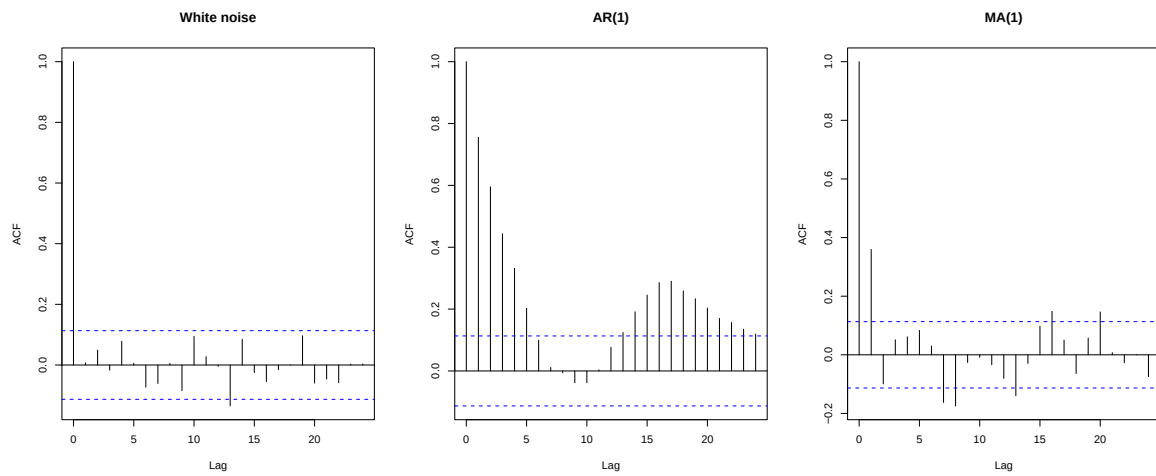


Figure 7: Three processes.

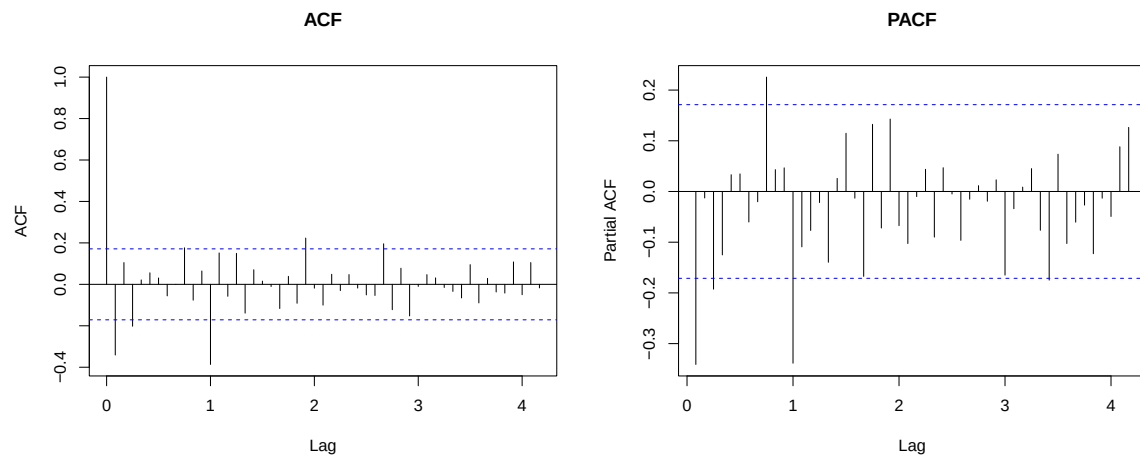


Figure 8: ACF and PACF plots for the AirPassengers dataset.

this is equivalent to: $\phi(L)y_t = c + \theta(L)\varepsilon_t$.

i Note

ARMA models are fitted by maximum likelihood due to the moving average part (not OLS).

The ARIMA(p, d, q) model

The ARIMA model applies d difference to y_t and then fits an ARMA(p, q) to the result.

$$\phi(L)(1 - L)^d y_t = c + \theta(L)\varepsilon_t$$

Table 2: Special cases of ARIMA.

Specification	Equivalent to
ARIMA(1,0,0)	AR(1)
ARIMA(0,0,1)	MA(1)
ARIMA(1,0,1)	ARMA(1,1)
ARIMA(0,1,0)	Random walk
ARIMA(0,1,0) with $c = 0$	Random walk with drift
ARIMA(1,1,1)	One difference, then ARMA(1,1)

Although Table 2 illustrates the special cases of ARIMA, sometimes there is a constant included that must be taken into account.

```

1      Ljung-Box test
2
3
4 data:  Residuals from ARIMA(1,1,1)
5 Q* = 231.49, df = 22, p-value < 2.2e-16
6
7 Model df: 2.    Total lags used: 24
1 NULL

```

In Figure 10 we can see that the residuals do not look like white noise, this is because the seasonality hasn't been removed from the data with an ARIMA(1,1,1) model.

The SARIMA(p, d, q)(P, D, Q) _{m} model

$$\Phi(L^m)\phi(L)(1 - L^m)^D(1 - L)^d y_t = \Theta(L^m)\theta(L)\varepsilon_t.$$

When we check an ACF plot of the `AirPassengers` series, we can see clear seasonality.

Figure 11 and Figure 10 suggest that we need to model the `AirPassengers` series with a seasonality component as well.

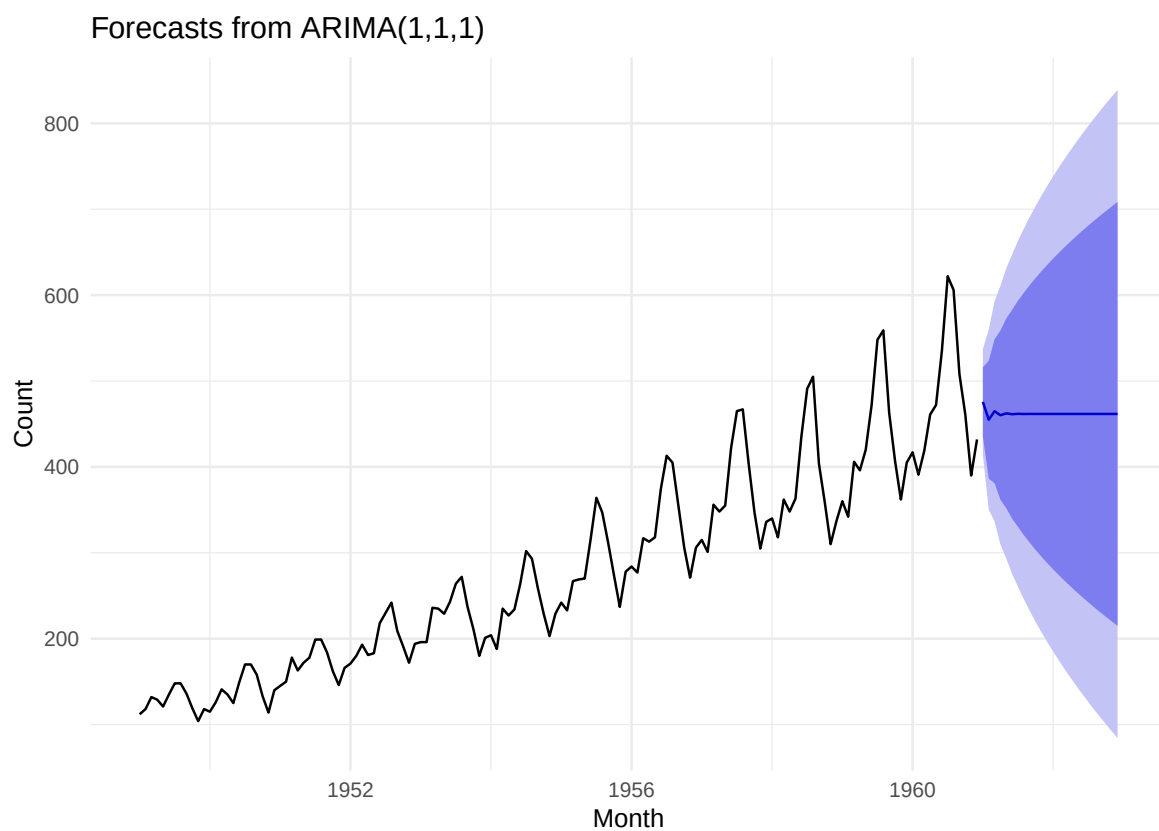


Figure 9: An ARIMA(1,1,1) model fitted to `AirPassengers` series with forecasts and prediction intervals for 24 months.

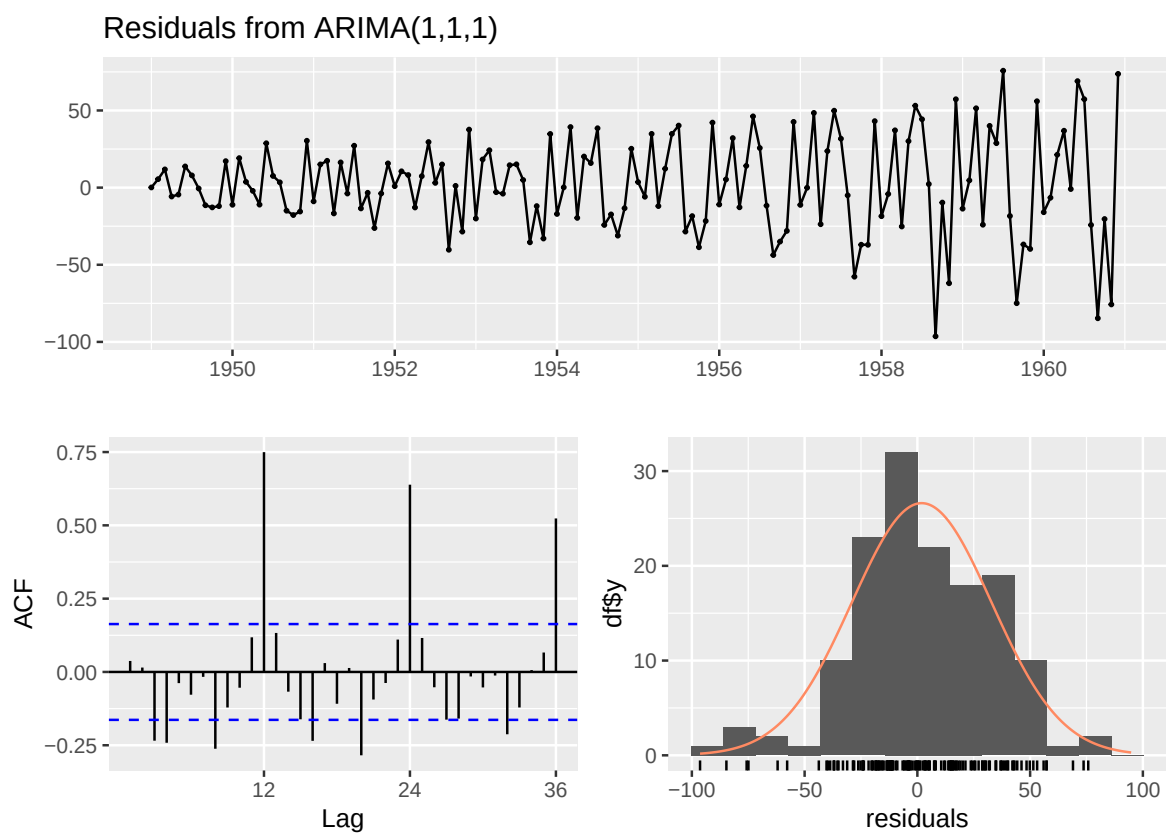


Figure 10: Residual diagnostics for the fitted ARIMA(1,1,1) model: the residual series, its ACF and a histogram against a normal density.

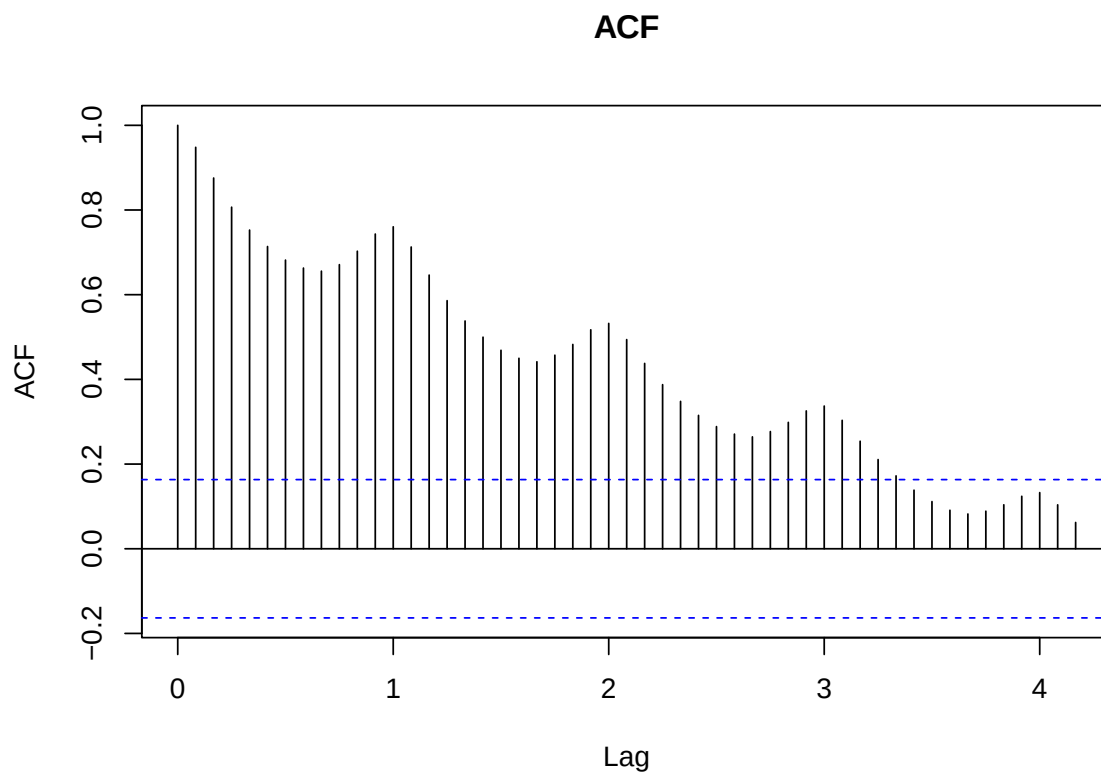


Figure 11: ACF plot of AirPassengers series to illustrate seasonality.

```

1 # Which SARIMA model is the best fit:
2 best_fit = auto.arima(AirPassengers,
3                       lambda=0,
4                       stepwise=FALSE,
5                       approximation=FALSE)
6 summary(best_fit)

1 Series: AirPassengers
2 ARIMA(0,1,1)(0,1,1)[12]
3 Box Cox transformation: lambda = 0
4
5 Coefficients:
6          ma1      sma1
7      -0.4018  -0.5569
8 s.e.    0.0896   0.0731
9
10 sigma^2 = 0.001371:  log likelihood = 244.7
11 AIC=-483.4   AICc=-483.21   BIC=-474.77
12
13 Training set error measures:
14              ME      RMSE      MAE      MPE      MAPE      MASE
15 Training set 0.05140418 10.15504 7.357553 -0.004079016 2.623636 0.229706
16              ACF1
17 Training set -0.03689673

```

Figure 12 carries the seasonal shape forward while the intervals widen. This is the behaviour a seasonal model is fitted for.

Conditional Variance Models

```

1 Tbill = Tbrate[,1]
2 Del.Tbill = diff(Tbill)

```

Problem 1

```

1 library(FinTS)
2 library(rugarch)
3
4 plot(Tbrate[,2],type='l',main='Raw data')

```

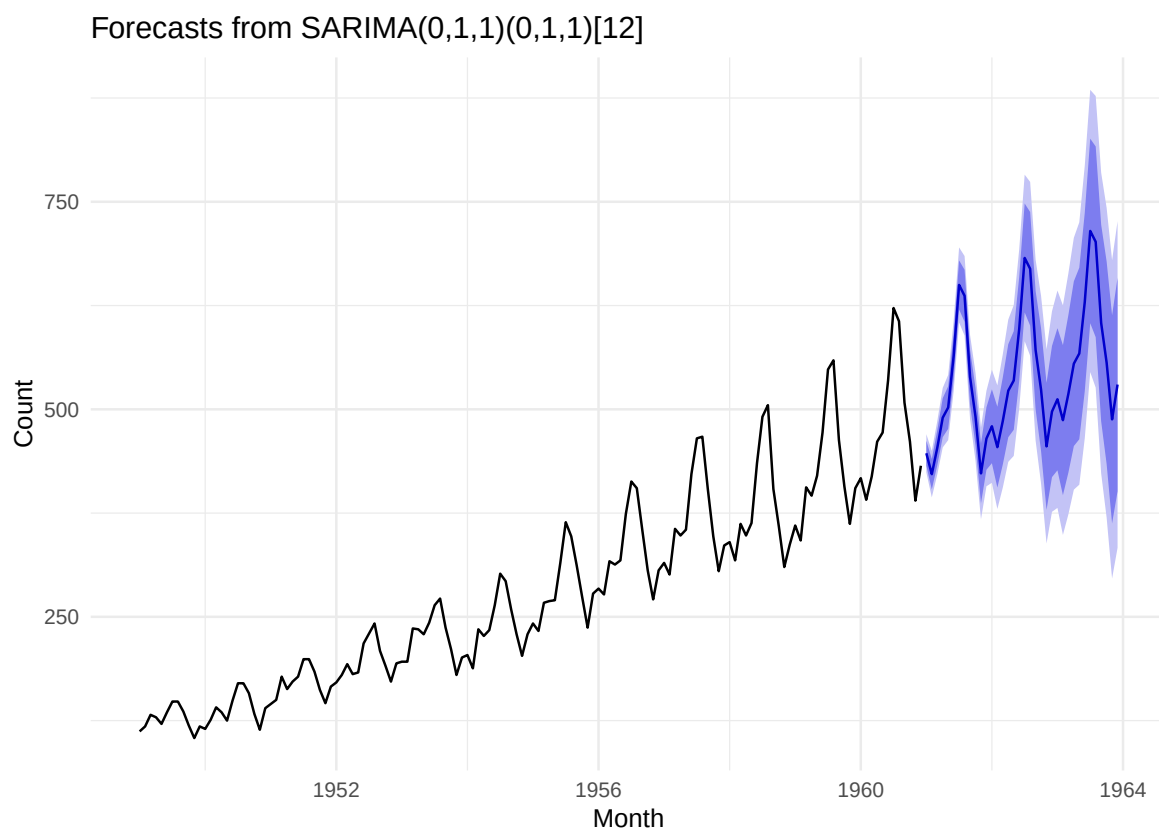
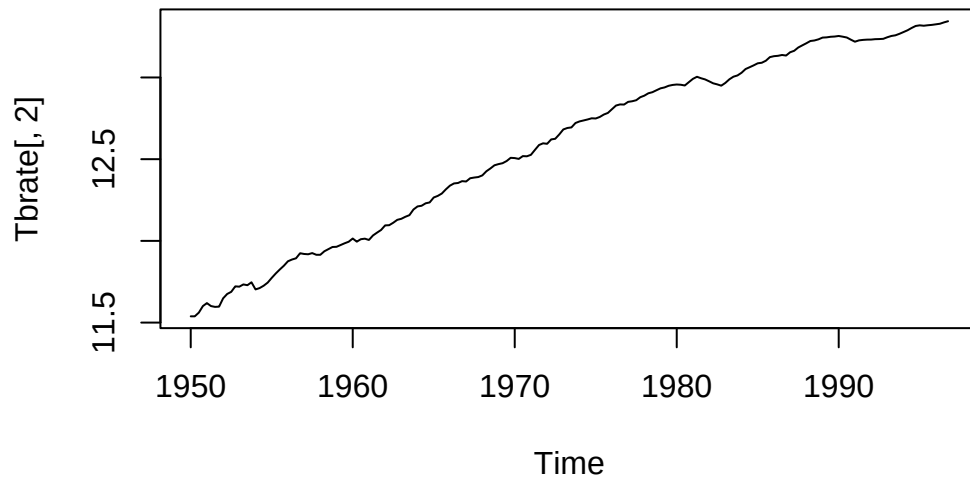


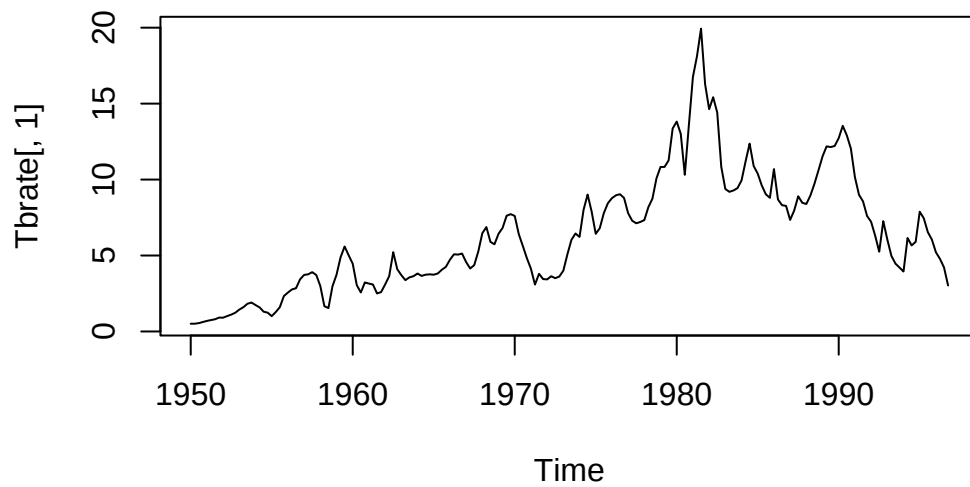
Figure 12: A SARIMA(1,1,1)(0,1,1) model fitted to AirPassengers series with a stochastic level and an evolving seasonal pattern, forecast three years ahead.

Raw data

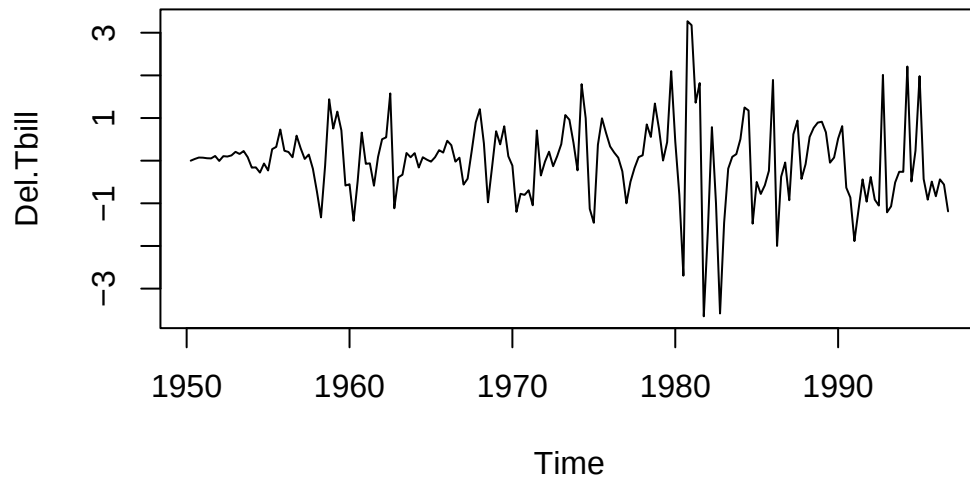


```
1 plot(Tbrate[,1],type='l',main='Returns')
```

Returns



```
1 plot(Del.Tbill)
```



Spectral Analysis

Section 2

Data Statement

[cite: 1]

Teaching Session Summaries

[cite: 1]

Worked Tutorials & Software Implementations

[cite: 1]

Topic Exploration

[cite: 1]

Projects

Overview

This section contains four substantial projects spanning Section 1 and Section 2[cite: 1].

Project	Section	Project Type	Focus Area	Link
Project 1	Section 1	Simulation Study	Model Comparison	Summary [cite: 1]
Project 2	Section 1	R Package Implementation	Computational Programming	Summary [cite: 1]
Project 3	Section 2	Shiny Application	Interactive Visualization	Summary [cite: 1]
Project 4	Section 2	Exploratory Data Analysis	Real-World Data	Summary [cite: 1]

Project 1

This is just basic EDA so far. I might only use this dataset to fit a hidden markov model once we've covered that topic in class.

- **Section & Topic:**[cite: 1]
- **Project Type:** (Implementation / Shiny App / Exploration / Simulation)[cite: 1]
- **Question / Aim:**[cite: 1]
- **Data Used:** (Include source details if real-world data)[cite: 1]
- **Methods Used:**[cite: 1]

Key Findings

[cite: 1]

Reflections

- **What I would do differently next time:**[cite: 1]

Links

- [Detailed Analysis & Code Script](#)[cite: 1]

```

1 #|echo: false
2 #|warning: false
3 # Load required libraries:
4 library(readr)
5 library(tidyverse)
6 library(tsibble)
7 library(lubridate)
8 library(dplyr)

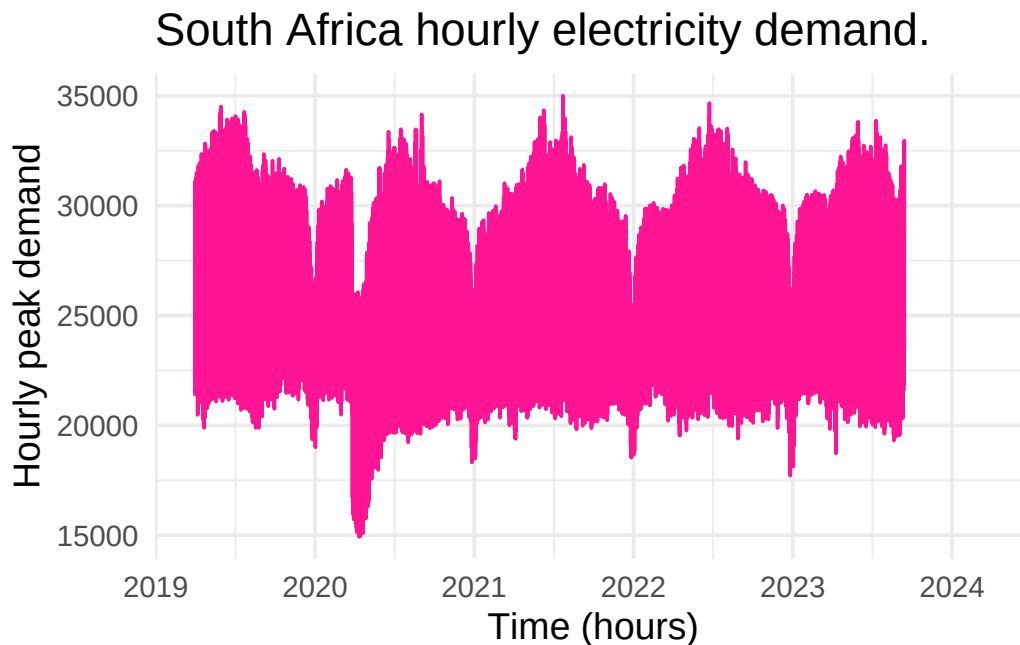
```

```

1 # Read in dataset:
2 dat = read_csv('_data/ESK6816.csv')
3 # View(dat)

```

```
1 # Convert data to time series object:
2 demand_ts = ts(
3   dat$`RSA Contracted Demand`,
4   start = c(2019, 2161), # Hour 2161 of 2019 (April 1st)
5   frequency = 8766      # Hourly frequency per year
6 )
7
8 # Convert time series object to dataframe (for plotting):
9 demand_df = data.frame(
10   hour = as.numeric(time(demand_ts)),
11   count = as.numeric(demand_ts)
12 )
13
14 # Plot:
15 ggplot(data = demand_df,
16   mapping = aes(x = hour,
17     y = count)) +
18   geom_line(col = 'deeppink') +
19   theme_minimal(base_size=14) +
20   labs(title = 'South Africa hourly electricity demand.',
21     x = 'Time (hours)',
22     y = 'Hourly peak demand')
```



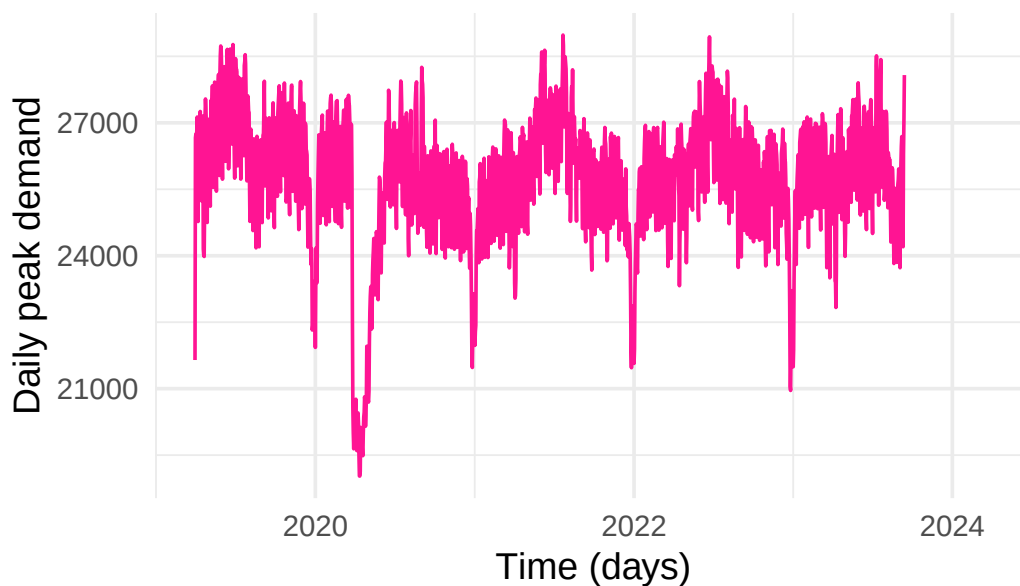
As seen in the above plot, we cannot get a clear idea of the seasonality. We need to aggregate the data.

To handle the leap years present in the data we will use tsibble.

```
1 # Create a tsibble dataframe:
2 df_ts = tibble(datetime=seq(from=as.POSIXct('2019-04-01 00:00:00'),
3   length.out=length(demand_ts),
```

```
4         by='hour'),
5         demand=as.numeric(demand_ts)
6       ) |>
7     as_tsibble(index=datetime)
8
9 # Aggregate to daily values:
10 demand_daily_ts = df_ts |>
11   index_by(date=~as.Date(.)) |>
12   summarise(demand_mean=mean(demand, na.rm=TRUE))
13
14 # Plot:
15 ggplot(data = demand_daily_ts,
16        mapping = aes(x = date,
17                      y = demand_mean)) +
18   geom_line(col = 'deeppink') +
19   theme_minimal(base_size=14) +
20   labs(title = 'South Africa daily electricity demand.',
21        x = 'Time (days)',
22        y = 'Daily peak demand')
```

South Africa daily electricity demand.



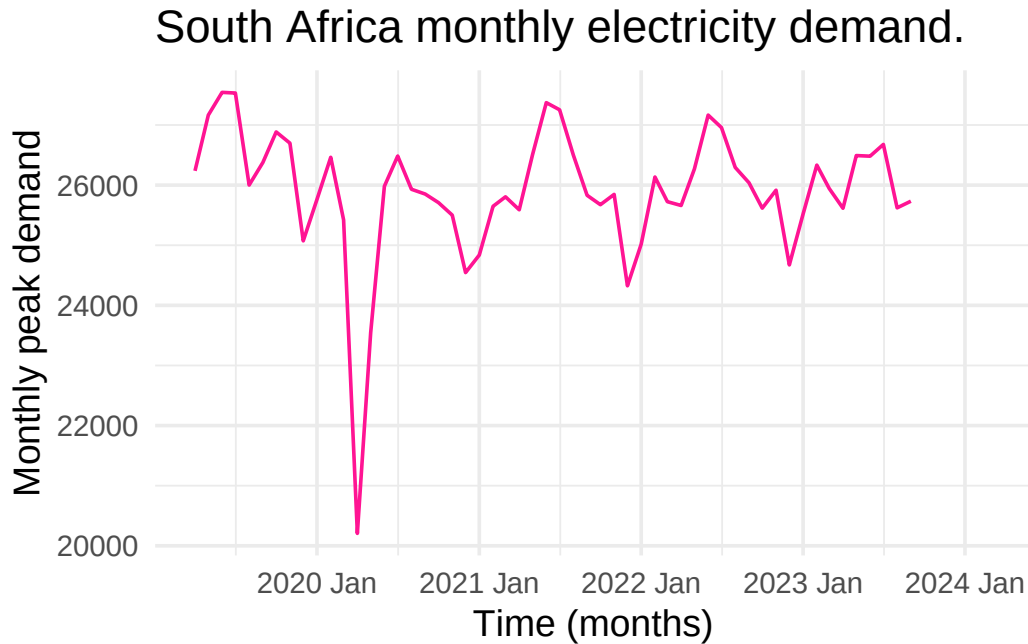
Again, there is still a lot of unnecessary noise in the data so we're going to aggregate to months.

```
1 # Aggregate to monthly values:
2 demand_monthly_ts = df_ts |>
3   index_by(year_month=~yearmonth(.)) |>
4   summarise(demand_mean=mean(demand, na.rm=TRUE))
5
6 # Plot:
7 ggplot(data = demand_monthly_ts,
8        mapping = aes(x = year_month,
```

```

9         y = demand_mean)) +
10 geom_line(col = 'deeppink') +
11 theme_minimal(base_size=14) +
12 labs(title = 'South Africa monthly electricity demand.',
13       x = 'Time (months)',
14       y = 'Monthly peak demand')

```



Project 2

The plan for this project is to fit all the autoregressive models to both datasets, compare them and forecast using the best models. I'm hoping to then use these same datasets in another projects to see if SST and historic dam levels can be used to predict drought in Southern Africa (I'm not 100% sure how I'm going to do this but I think it'll be interesting)

Purely for interests sake, here is a website that maps the sea surface temperature anomalies: https://earth.nullschool.net/#current/ocean/surface/currents/overlay=sea_surface_temp_anom 100.92,9.72,366/loc=-99.041,-1.152

SST along the equatorial band in the eastern and central Pacific is much higher than past temperatures. This is aligning with experts predicting a super El Nino in 2026/2027.

- **Section & Topic:** Autoregressive Models
- **Project Type:** Model implementaion
- **Question / Aim:** Is sea surface temperature in the eastern Pacific Ocean and Theewaterskloof dam levels predictable?

- **Data Used:** [<https://www.pmel.noaa.gov/tao/drupal/disdel/>] [<https://odp-cctegis.opendata.arcgis.com/>] I will reference this properly!
- **Methods Used:** AR,MA,ARMA,ARIMA,SARIMA,SARIMAX (at least this is the plan)

Key Findings

[cite: 1]

Reflections

- **What I would do differently next time:**[cite: 1]

Links

- [Detailed Analysis & Code Script](#)[cite: 1]

Predicting drought in Southern Africa

The aim of this project is to predict Theewaterskloof dam levels as well changes in sea surface temperatures in the eastern Pacific Ocean (along 95W) and use this data to predict drought occurrence in Southern Africa.

The El Nino phase of the El Nino Southern Oscillation (ENSO) is characterise by anomalous warming of sea surface temperatures in the eastern and central Pacific Ocean. This phenomenon typically starts between March and June, reaching its peak in December. An El Nino event causes warm and dry conditions over Southern Africa, typically taking effect 3-6 months after the onset of El Nino, in the Pacific. Over the 2015-2016 season, the El Nino weather patterns resulted in prolonged drought in Southern Africa with dam levels dropping and severe water restrictions being implemented.

The South African Weather Service (SAWS) is warning against a super El nino event in 2026-2027, as current sea surface temperatures rise beyond the threshold used to define a very strong El Nino.

We are interested in the Theewaterskloof dam current levels as Theewaterskloof dam is the main supplier of water to Cape Town.

References:

[https://onlinelibrary.wiley.com/doi/abs/10.1002/env.476?casa_token=Eg_zhVj43usAAAAA%3Ar0Wj10YIV8mbMZnQvBcHiz9Cc_yBXPmfHGftiHsExL1riAtWaHZKw]

[[https://www.thelancet.com/journals/lancet/article/PIIS0140-6736\(03\)12421-X/fulltext](https://www.thelancet.com/journals/lancet/article/PIIS0140-6736(03)12421-X/fulltext)]

[<https://mg.co.za/the-green-guardian/2026-09-01-the-saws-warns-exceptionally-strong-2026-2027-el-nino-could-bring-severe-heatwaves-to-sa/>]

```

1 # A tibble: 1 x 1
2   n_missing
3   <int>
4 1         1

```

Exploratory Data Analysis

The duplicate observation in the `Dam_Levels_from_2000` dataset was removed by retaining only the first occurrence of each date, using `distinct(DATE, .keep_all=TRUE)`. The dam level series contained only 1 missing value that was linearly interpolated.

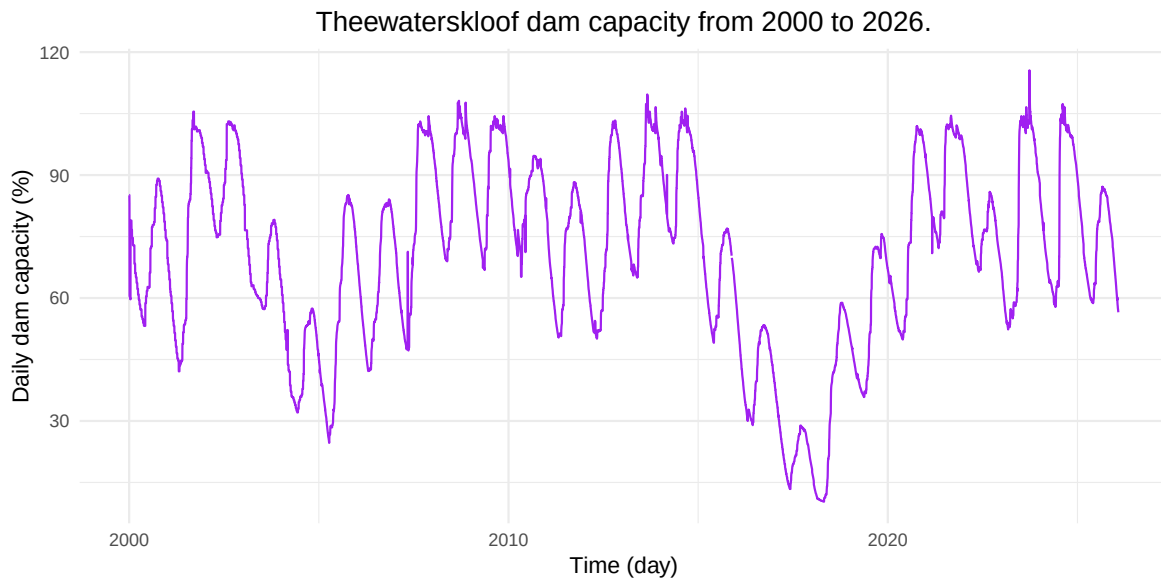


Figure 13: Theewaterskloof dam capacity from January 2000 to January 2026.

As seen in both Figure 13 and Figure 14 there is noise in the daily observations. The daily data is aggregated to monthly averages to obtain a clearer visualisation of the seasonality present in the data and to smooth out high frequency noise.

As seen in Figure 15 and Figure 16, both datasets display strong 12-month seasonal patterns.

Both the `Dam_Levels_from_2000` dataset and the `sst` dataset have observations from 2000 to 2026. The sea surface temperature series has a period of missing observations from November 2021 to March 2023 due to the buoy array not recording observations. This gap cannot be reliably smoothed over and thus has been left as missing.

As the forecast horizon grows, uncertainty in the predictions increases. Experts are predicting an exceptionally strong El Nino event in 2026/2027 and since both series extend to the present year it will be interesting to see if the fitted models pick up the El Nino phenomenon as it develops.

Southern Africa experienced severe droughts in 2015/2016 and 2018/2019. Both of

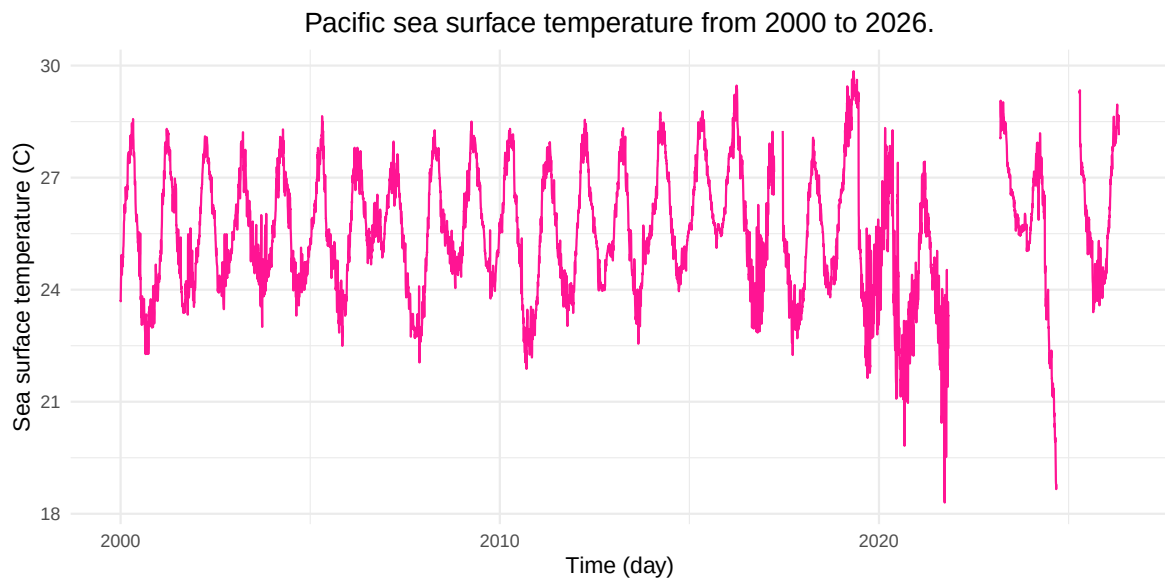


Figure 14: Average Pacific sea surface temperature from January 2000 to April 2026.

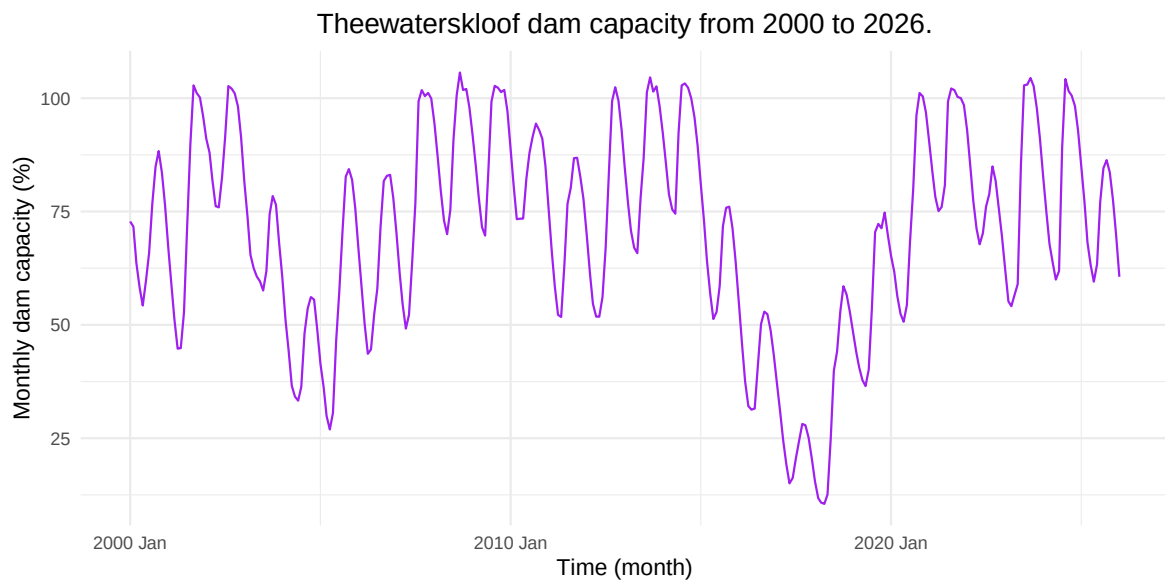


Figure 15: Theewaterskloof dam capacity from January 2000 to January 2026.

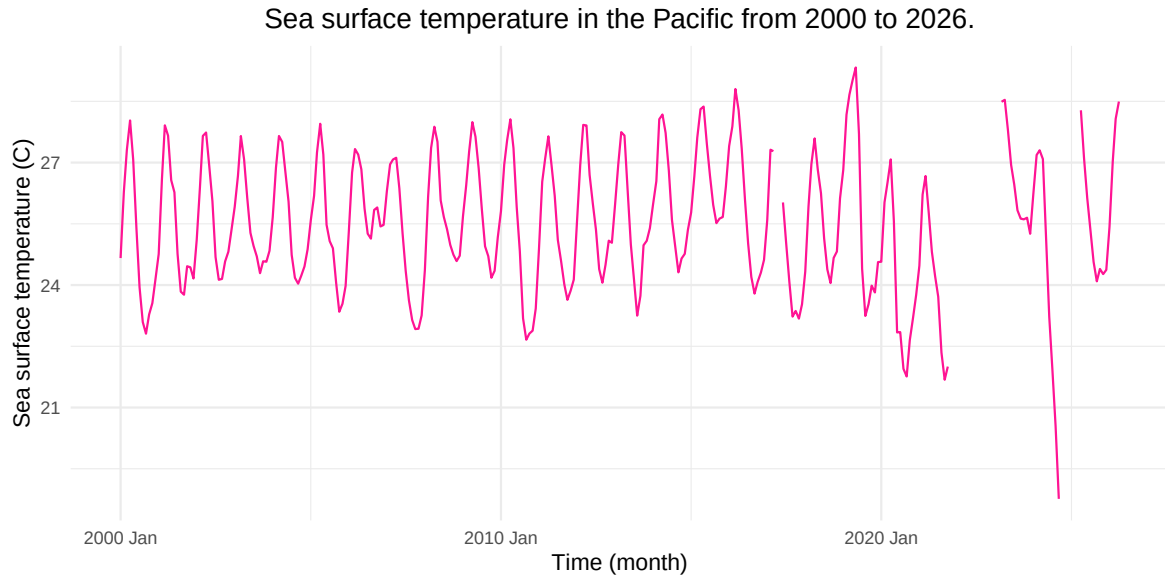


Figure 16: Average Pacific sea surface temperature from January 2000 to April 2026.

these periods saw significant El Nino conditions which supports the idea that eastern Pacific sea surface temperature can be used to predict Theewaterskloof dam levels.

```

1 # # label: fig_15161819
2 # # fig-cap: "Theewaterskloof dam capacity fro 2015-2016 and 2018-2019."
3 # # fig-width: 12
4 # # fig-height: 5
5 # # echo: false
6 # # message: false
7 # # warning: false
8 #
9 # damlvl_2015_2016 = damlvl_ts |>
10 #   filter(
11 #     DATE >= ymd('2015-01-01') & DATE <= ymd('2016-12-31')
12 #   )
13 # damlvl_2018_2019 = damlvl_ts |>
14 #   filter(
15 #     DATE >= ymd('2018-01-01') & DATE <= ymd('2019-12-31')
16 #   )
17 #
18 # # Plot:
19 # plot_20152016 = ggplot(data = damlvl_2015_2016,
20 #   mapping = aes(x = DATE,
21 #     y = THEEWATERSKLOOF_Current)) +
22 #   geom_line(col = 'purple') +
23 #   theme_minimal(base_size=14) +
24 #   labs(title = 'Theewaterskloof dam capacity for 2015-2016.',
25 #     x = 'Time (month)',
26 #     y = 'Monthly dam capacity (%)') +
27 #   theme(
28 #     plot.title = element_text(hjust = 0.5)
29 #   )
30 #
31 # # Plot:

```

```

30 # plot_20182019 = ggplot(data = damlvl_2018_2019,
31 #   mapping = aes(x = DATE,
32 #   y = THEEWATERSKLOOF_Current)) +
33 #   geom_line(col = 'purple') +
34 #   theme_minimal(base_size=14) +
35 #   labs(title = 'Theewaterskloof dam capacity for 2018-2019.',
36 #   x = 'Time (month)',
37 #   y = 'Monthly dam capacity (%)') +
38 #   theme(
39 #     plot.title = element_text(hjust = 0.5)
40 #   )
41 #
42 # plot_20152016 + plot_20182019

```



Figure 17: Theewaterskloof dam capacity for 2015-2016 and 2018-2019, alongside sea surface temperature from six months earlier (each panel on its own true observation dates).

Sea surface temperatures and dam levels both show strong seasonal patterns. Dam levels dip around April/May and peak around August. Sea surface temperatures peak around February/March and drop around September/October.

It is worth noting that the structural gaps present in the sea surface temperature series result in some months having more observations than others.

Need to mention autocorrelation here.

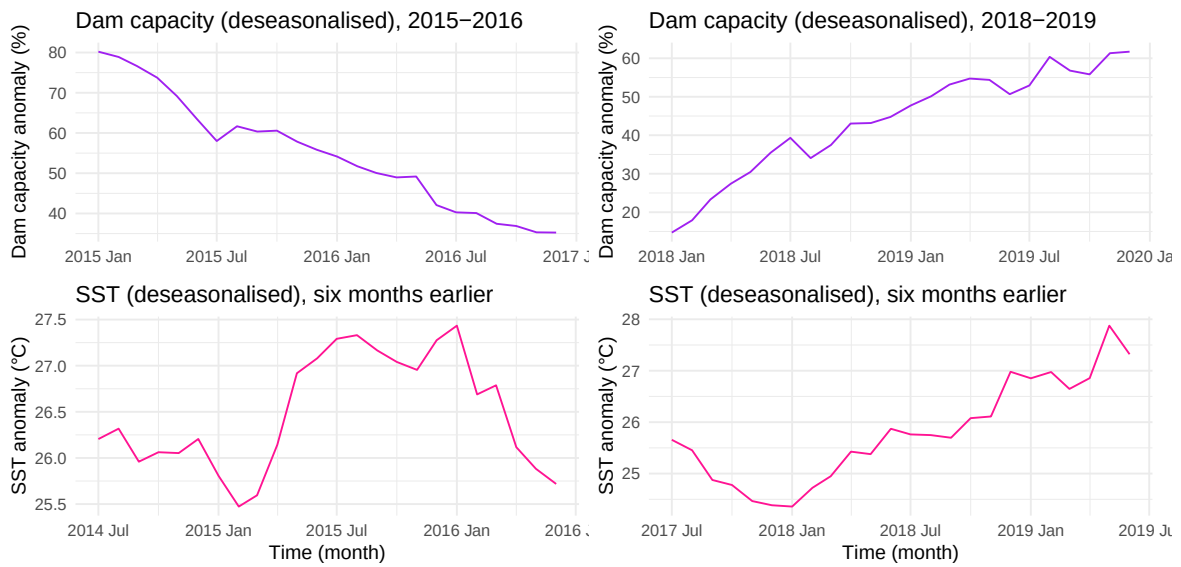


Figure 18: Deseasonalised lag comparison of dam level and sea surface temperature.

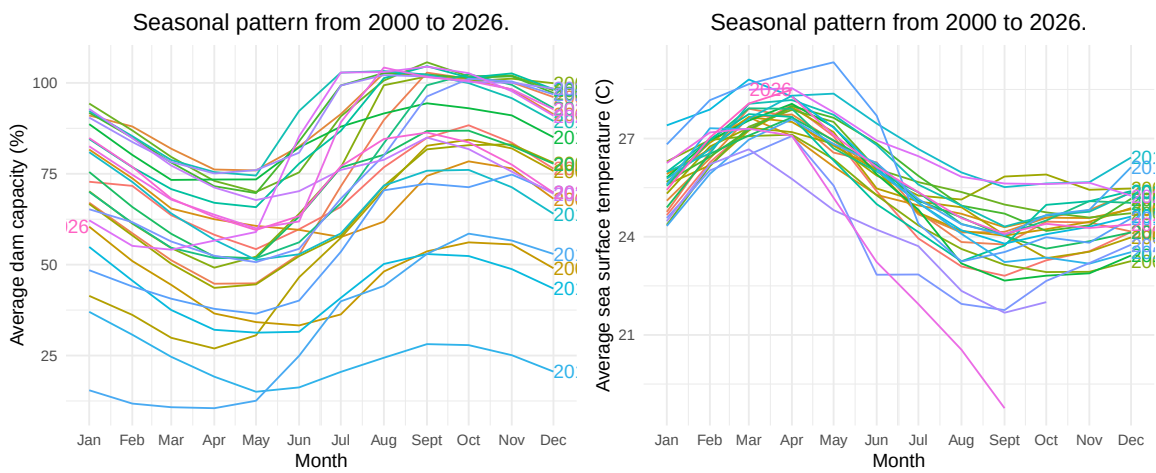


Figure 19: Seasonal pattern of Theewaterskloof dam capacity and average Pacific sea surface temperature.

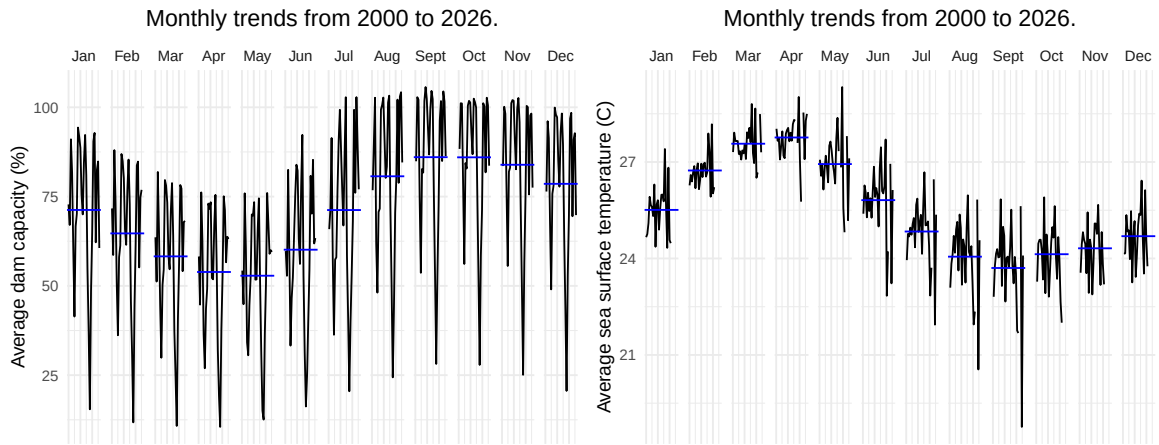


Figure 20: Monthly trends in Theewaterskloof dam capacity and average Pacific sea surface temperature.

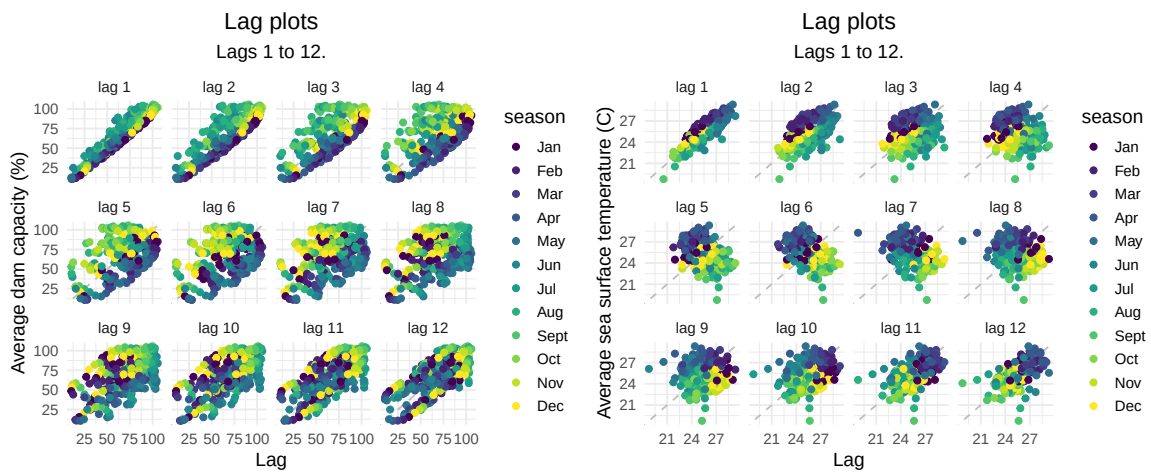


Figure 21: Lag plots of Theewaterskloof dam capacity and average Pacific sea surface temperature.

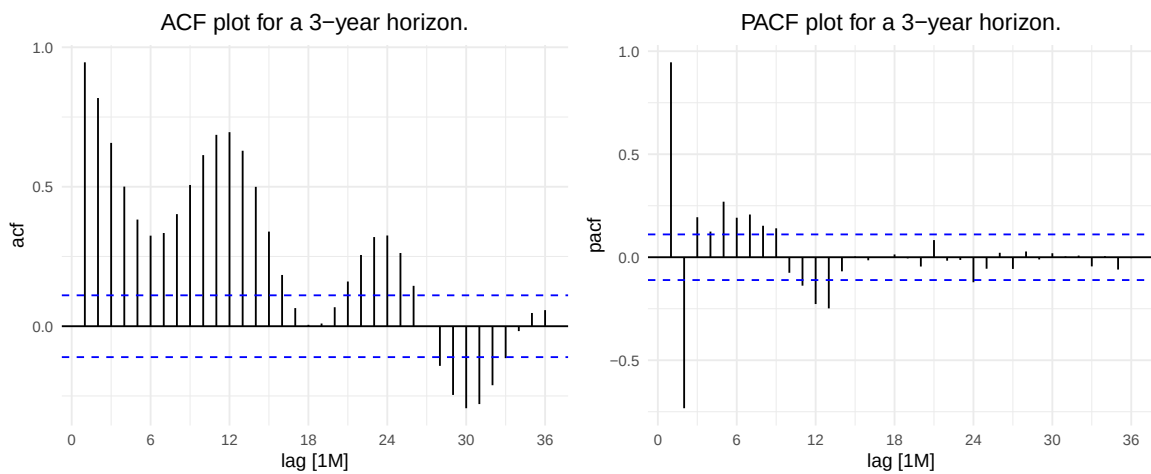


Figure 22: ACF and PACF plots for Theewaterskloof dam capacity.

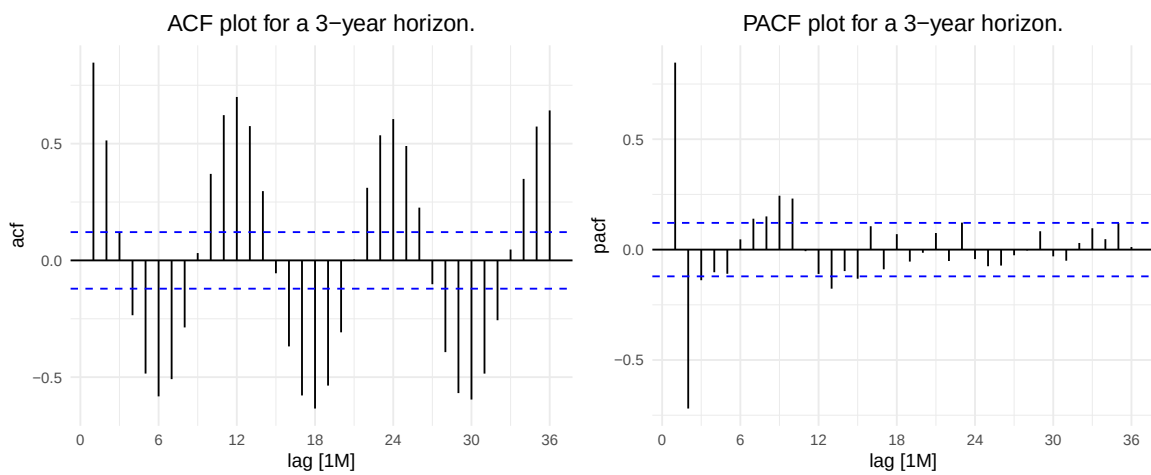


Figure 23: ACF and PACF plots for Pacific sea surface temperature.

Both Figure 22 and Figure 23 indicate that these time series are non-stationary. The ACF of the sea surface temperature illustrates a regular seasonal pattern with sharp peaks occurring at regular intervals compared to the ACF of dam levels which has a less stable seasonal pattern seen by the less regularly-spaced peaks. This suggests that the dam level pattern is influenced by a long-term irregular hydrological cycle as opposed to seasonal rainfall patterns. Both PACF plots have a large spike at lag 1. This is typical of a non-stationary process. The dam level series has some weaker spikes around lag 6-8 that indicate short-run structure but the sea surface temperature PACF is mostly within the bounds after lag 1. The order selection for the AR and SARIMA models is based on the ACF and PACF plots of the seasonally differenced series.

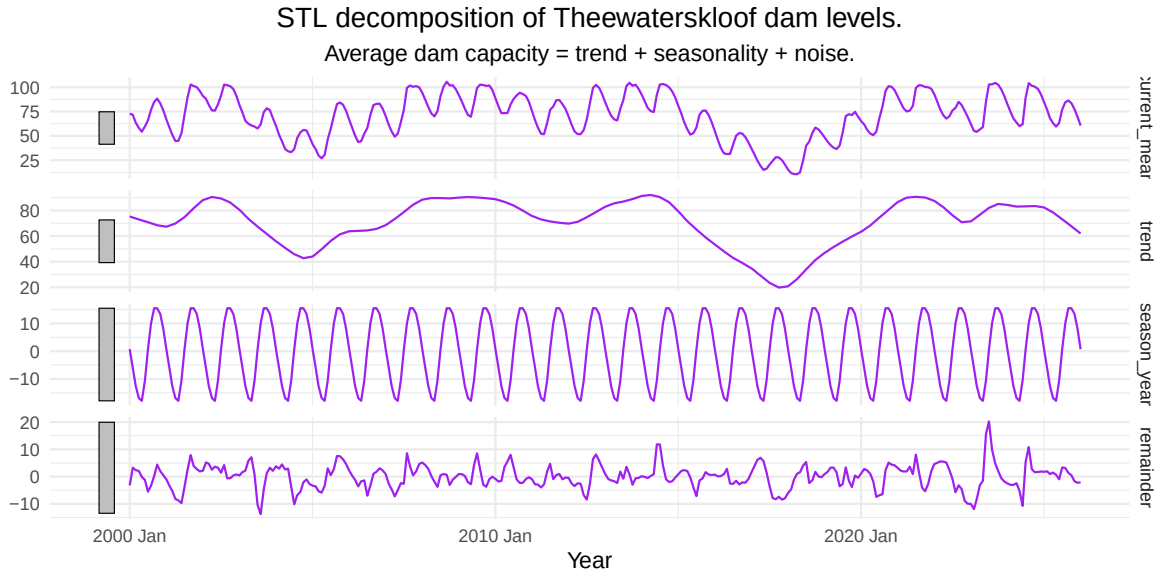


Figure 24: Decomposition of Theewaterskloof dam capacity series.

Figure 25 plots the decomposition of the sea surface temperature series. There are missing observations from November 2021 to March 2023 and October 2024 to March 2025. These observations have been interpolated not observed for completeness of the decomposition across the entire series and should not be interpreted as true measurements.

As seen in Figure 24 and Figure 25 both the dam level and sea surface temperature series show strong seasonal trends with Figure 24 showing multi-year drought and recovery cycles. There are noticeable dips in 2005 and 2018, as opposed to a stable long-run trend. The SST trend observed in the sea surface temperature series is more stable with a slightly smoother trend. This can be attributed to sea surface temperatures being driven by the annual warming and cooling due to heating angles whereas dam levels are driven not only by seasonal patterns by also by the long-term irregular hydrological cycle.

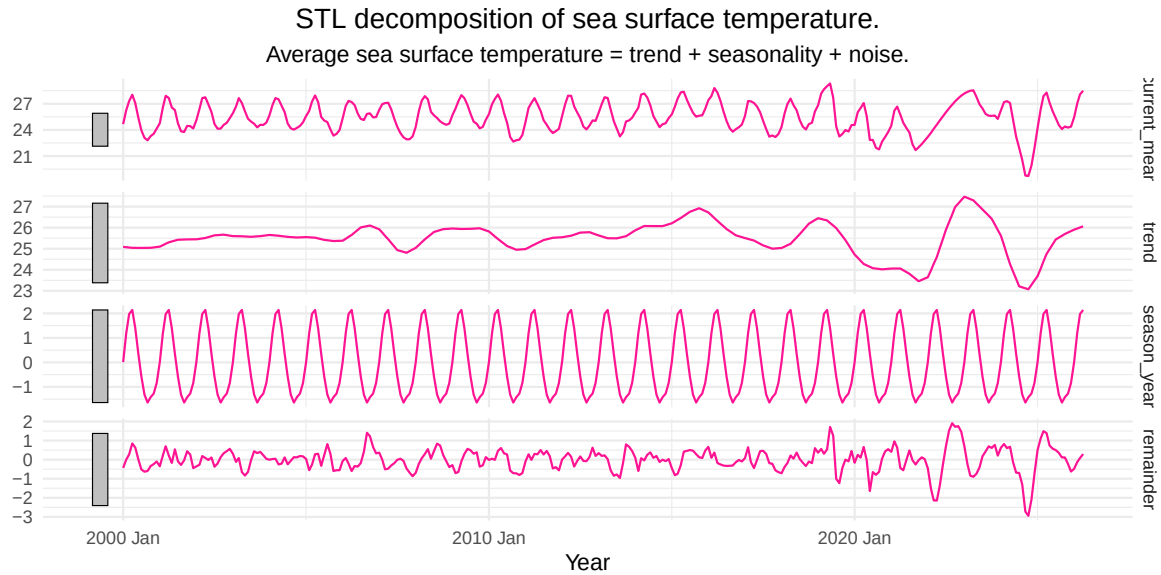


Figure 25: Decomposition of average Pacific sea surface temperature series from 2000 to 2026.

Mean	Naive	Seasonal naive	Drift
24.945	7.521	17.308	7.65

The naive model produces the smallest residuals when compared to the other benchmark models that were fit.

```
1 # A tibble: 1 x 1
2   nsdiffs
3   <int>
4 1       1
```

```
1 # A tibble: 1 x 1
2   nsdiffs
3   <int>
4 1       0
```

```
1 # A tibble: 1 x 1
2   nsdiffs
3   <int>
4 1       1
```

```
1 # A tibble: 1 x 1
2   nsdiffs
3   <int>
4 1       0
```

We only need to seasonally difference the `Dam_Levels_from_2000` and `sst` datasets, no linear differencing is required.

Model fitting

This project fits and compares AR,MA,ARMA,ARIMA and SARIMA against the benchmark models: Mean, Naive, Seasonal naive and Drift, for both the `Dam_Levels_from_2000` and `sst` datasets. A time series regression model is then fit on `Dam_Levels_from_2000` using the sea surface temperature as a predictor to determine if sea surface temperature in the eastern Pacific can improve the prediction of Theewaterskloof dam levels to aid in drought prediction.

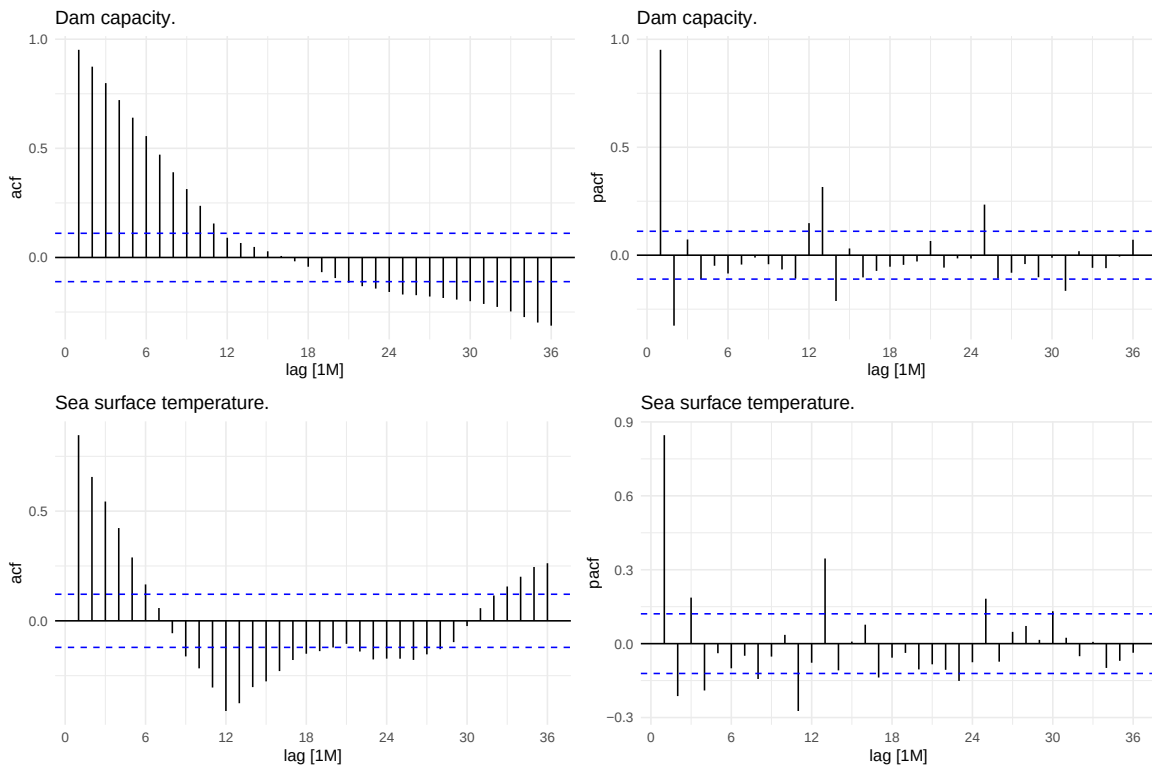


Figure 26: ACF and PACF plots for both dam levels and sea surface temperature for the seasonally differenced data.

@

```
1 # Dam: full series available to 2026, hold out the last 2 years as test
2 dam_train = monthly_damlvl_ts |> filter_index(. ~ "2023 Dec")
3 dam_test   = monthly_damlvl_ts |> filter_index("2024 Jan" ~ .)
4
5 # SST: use the clean pre-2021 segment (consistent with your unit-root/ACF
6   work),
7 # hold out the last ~2 years of that clean stretch
8 sst_train = monthly_ts_pre2021 |> filter_index(. ~ "2019 Dec")
9 sst_test  = monthly_ts_pre2021 |> filter_index("2020 Jan" ~ .)
10
11 dam_fit = dam_train |>
12   model(
13     mean = MEAN(current_mean),
14     naive = RW(current_mean),
```

```

5   snaive = SNAIVE(current_mean),
6   drift  = RW(current_mean ~ drift()),
7   ar     = ARIMA(current_mean ~ pdq(1,0,0) + PDQ(0,1,0)),
8   ma     = ARIMA(current_mean ~ pdq(0,0,1) + PDQ(0,1,0)),
9   arma   = ARIMA(current_mean ~ pdq(1,0,1) + PDQ(0,1,0)),
10  sarima = ARIMA(current_mean ~ pdq(1,0,0) + PDQ(1,1,0)),
11  auto   = ARIMA(current_mean)
12 )
13
14 dam_fc = dam_fit |> forecast(new_data = dam_test)
15
16 dam_accuracy = dam_fc |>
17   accuracy(monthly_damlvl_ts,
18     measures = list(point_accuracy_measures,
19       distribution_accuracy_measures))
19
20 knitr::kable(dam_accuracy, digits = 3)

```

.model	.type	ME	RMSE	MAE	MPE	MAPE	MASER	RMSS	AACF1	percentile	CRPS
ar	Test	-	29.244	24.587	-	33.246	1.579	1.605	0.795	17.088	16.921
		24.587			33.246						
arma	Test	-	24.536	20.176	-	27.346	1.296	1.346	0.774	13.532	13.399
		20.100			27.246						
auto	Test	1.173	7.061	5.740	0.515	7.276	0.369	0.387	0.713	4.911	4.864
drift	Test	-	19.590	16.836	-	24.383	1.081	1.075	0.774	12.097	11.978
		13.694			21.284						
ma	Test	-	14.809	12.188	-	16.576	0.783	0.813	0.757	8.442	8.359
		4.042			5.398						
mean	Test	8.134	16.144	13.067	7.561	15.544	0.839	0.886	0.776	9.593	9.499
naive	Test	-	18.977	16.266	-	23.542	1.045	1.041	0.776	11.756	11.641
		12.871			20.186						
sarima	Test	-	14.855	11.578	-	15.990	0.744	0.815	0.794	7.917	7.840
		9.287			13.483						
snaive	Test	-	15.560	12.768	-	17.006	0.820	0.854	0.737	9.254	9.164
		2.300			3.053						

```

1 # AIC/BIC only meaningful within the ARIMA family (same d, D) -- excludes
2 # the benchmark models, which aren't fit by the same likelihood
3 dam_fit |>
4   select(ar, ma, arma, sarima, auto) |>
5   glance() |>
6   knitr::kable(digits = 2)

```

.model	logLik	AIC	BIC	ar_roots	ma_roots
ar	26.33	1691.64	1698.67	0.42843+0i	
				843.72	

model	MAICBI	ar_roots	ma_roots
ma	99.36 20528098000.19 1027.47		-1.016376+0i
arn	22.53 16496749682.57 821.86	2577+0i	-2.489425+0i
sarima	21.20 163563545.72 814.73	35235+7.535235e-01i, -0.7535235+7.535235e-01i, -0.2758087-1.029332e+00i, 0.2758087+1.029332e+00i, -1.0293323+2.758087e-01i, -0.7535235-7.535235e-01i, 1.0274617+3.908262e-15i, -0.2758087+1.029332e+00i, -1.0293323-2.758087e-01i, 0.2758087-1.029332e+00i, 1.0293323-2.758087e-01i, 1.0293323+2.758087e-01i, 0.7535235-7.535235e-01i	
auto	13.45 1529509541.16 757.51	78373+0.02943642i, 1.1078373-0.02943642i, 0.9976095-2.35916622i, 0.9976095+2.35916622i	5.052511e-01+8.751206e-01i, -8.751206e-01+5.052511e-01i, -5.052511e-01-8.751206e-01i, 8.751206e-01-5.052511e-01i, 1.035264e-16+1.010502e+00i, -1.010502e+00+5.222335e-16i, -9.933165e-16-1.010502e+00i, 1.010502e+00-3.578530e-15i, -5.052511e-01+8.751206e-01i, -8.751206e-01-5.052511e-01i, 5.052511e-01-8.751206e-01i, 8.751206e-01+5.052511e-01i, 1.375095e+00+6.749240e-15i

```

1 sst_fit = sst_train |>
2   model(
3     mean    = MEAN(current_mean),
4     naive   = RW(current_mean),
5     snaive  = SNAIVE(current_mean),
6     drift   = RW(current_mean ~ drift()),
7     ar      = ARIMA(current_mean ~ pdq(1,0,0) + PDQ(0,1,0)),
8     ma      = ARIMA(current_mean ~ pdq(0,0,1) + PDQ(0,1,0)),
9     arma    = ARIMA(current_mean ~ pdq(1,0,1) + PDQ(0,1,0)),
10    sarima   = ARIMA(current_mean ~ pdq(1,0,0) + PDQ(1,1,0)),
11    auto     = ARIMA(current_mean)
12  )
13
14 sst_fc = sst_fit |> forecast(new_data = sst_test)

```

```

15
16 sst_accuracy = sst_fc |>
17   accuracy(monthly_ts_pre2021,
18     measures = list(point_accuracy_measures,
19       distribution_accuracy_measures))
19
20 knitr::kable(sst_accuracy, digits = 3)

```

.model	.type	ME	RMSE	MAE	MPE	MAPE	MASE	RMSE	MAE	ACF1	percentile	CRPS
ar	Test	-	1.785	1.420	-	5.898	1.829	1.779	0.412	1.044	1.035	
		1.420			5.898							
arma	Test	-	1.990	1.642	-	6.833	2.116	1.983	0.438	1.195	1.184	
		1.642			6.833							
auto	Test	-	1.808	1.667	-	7.071	2.148	1.802	0.527	1.297	1.285	
		1.667			7.071							
drift	Test	-	1.773	1.545	-	6.531	1.991	1.767	0.786	1.115	1.104	
		0.438			2.329							
ma	Test	-	2.376	2.073	-	8.551	2.671	2.368	0.468	1.604	1.590	
		2.073			8.551							
mean	Test	-	2.289	1.918	-	8.333	2.472	2.282	0.786	1.406	1.392	
		1.512			6.805							
naive	Test	-	1.775	1.547	-	6.540	1.993	1.769	0.786	1.107	1.096	
		0.442			2.349							
sarima	Test	-	2.025	1.818	-	7.615	2.342	2.018	0.416	1.373	1.361	
		1.818			7.615							
snaive	Test	-	2.440	2.158	-	8.898	2.781	2.432	0.437	1.610	1.596	
		2.158			8.898							

```

1 sst_fit |>
2   select(ar, ma, arma, sarima, auto) |>
3   glance() |>
4   knitr::kable(digits = 2)

```

.model	logLik	AIC	BIC	Gr_roots	ma_roots
ar	0.27	354.92	361.76	1.768917+0i	
		175.46			
ma	0.41	447.10	451.94		-1.193003+0i
		221.54			
arma	0.25	332.38	341.15	1.52228+0i	-2.366284+0i
		163.43			


```

1 polyroot(c(1, -0.7272)) # non-seasonal MA root
1 [1] 1.375138+0i

1 dam_fit2 = dam_train |>
2   model(
3     sarima_p1 = ARIMA(current_mean ~ pdq(1,0,0) + PDQ(1,1,0)), # your
4     original manual read
5     sarima_p2 = ARIMA(current_mean ~ pdq(2,0,0) + PDQ(0,1,1)),
6     sarima_p3 = ARIMA(current_mean ~ pdq(3,0,0) + PDQ(0,1,1)),
7     sarima_p4 = ARIMA(current_mean ~ pdq(4,0,0) + PDQ(0,1,1)), # auto's
8     auto      = ARIMA(current_mean)
9   )
10 dam_fc2 = dam_fit2 |> forecast(new_data = dam_test)
11
12 dam_fc2 |>
13   accuracy(monthly_damlvl_ts, measures = list(point_accuracy_measures,
14     distribution_accuracy_measures)) |>
14   knitr::kable(digits = 3)

```

.model	.type	ME	RMSE	MAE	MPE	MAPE	MASER	MSSA	ACF1	percentile	CRPS
auto	Test	1.173	7.061	5.740	0.515	7.276	0.369	0.387	0.713	4.911	4.864
sarima_p1	Test	-	14.855	11.578	-	15.990	0.744	0.815	0.794	7.917	7.840
		9.287			13.483						
sarima_p2	Test	-	7.484	6.091	-	7.988	0.391	0.411	0.725	4.994	4.946
		1.748			3.357						
sarima_p3	Test	-	7.670	6.207	-	8.214	0.399	0.421	0.725	5.013	4.964
		2.292			4.074						
sarima_p4	Test	-	7.538	6.125	-	8.052	0.393	0.414	0.725	5.035	4.986
		1.911			3.573						

```

1 dam_fit2 |> glance() |> knitr::kable(digits = 2)

```

.model	logAIC	logBIC	ma_roots	ma_roots
sarima2p1635636346.72	214.20	1635636346.72	-0.7535235+7.535235e-01i, -0.2758087-1.029332e+00i, 0.2758087+1.029332e+00i, -1.0293323+2.758087e-01i, -0.7535235-7.535235e-01i, 1.0274617+3.908262e-15i, -0.2758087+1.029332e+00i, -1.0293323-2.758087e-01i, 0.2758087-1.029332e+00i, 1.0293323-2.758087e-01i, 1.0293323+2.758087e-01i, 0.7535235-7.535235e-01i	
sarima2p1635636346.72	214.20	1635636346.72		5.067280e-01+8.776786e-01i, -8.776786e-01+5.067280e-01i, -5.067280e-01-8.776786e-01i, 8.776786e-01-5.067280e-01i, 7.886302e-17+1.013456e+00i, -1.013456e+00-1.259849e-14i, 1.324044e-14-1.013456e+00i, 1.013456e+00+1.069647e-14i, -5.067280e-01+8.776786e-01i, -8.776786e-01-5.067280e-01i, 5.067280e-01-8.776786e-01i, 8.776786e-01+5.067280e-01i
sarima4p43024084001.95	144.94	43024084001.95	3.197276+2.237974e+00i, 3.197276-2.237974e+00i	5.061894e-01+8.767458e-01i, -8.767458e-01+5.061894e-01i, -5.061894e-01-8.767458e-01i, 8.767458e-01-5.061894e-01i, -2.611509e-17+1.012379e+00i, -1.012379e+00-1.261335e-14i, 1.293492e-14-1.012379e+00i, 1.012379e+00+1.099314e-14i, -5.061894e-01+8.767458e-01i, -8.767458e-01-5.061894e-01i, 5.061894e-01-8.767458e-01i, 8.767458e-01+5.061894e-01i

.model	logLik	AIC	AICc	BIC	ar_roots	ma_roots
sarima_p4	14.08	4026.81	4027.12	4048.53	1.3909-0.4970i, -0.1230387+3.244173e+00i, -0.1230387-3.244173e+00i, 2.2099509+0.000000e+00i	5.065219e-01+8.773217e-01i, -8.773217e-01+5.065219e-01i, -5.065219e-01-8.773217e-01i, 8.773217e-01-5.065219e-01i, 7.047274e-17+1.013044e+00i, -1.013044e+00-1.256684e-14i, 1.269695e-14-1.013044e+00i, 1.013044e+00+1.080034e-14i, -5.065219e-01+8.773217e-01i, -8.773217e-01-5.065219e-01i, 5.065219e-01-8.773217e-01i, 8.773217e-01+5.065219e-01i
auto	13.45	1529.52	1554.16	1578.37	1.3909-0.4970i, -0.1230387+3.244173e+00i, -0.1230387-3.244173e+00i, 2.2099509+0.000000e+00i	5.052511e-01+8.751206e-01i, -8.751206e-01+5.052511e-01i, -5.052511e-01-8.751206e-01i, 8.751206e-01-5.052511e-01i, 1.035264e-16+1.010502e+00i, -1.010502e+00+5.222335e-16i, -9.933165e-16-1.010502e+00i, 1.010502e+00-3.578530e-15i, -5.052511e-01+8.751206e-01i, -8.751206e-01-5.052511e-01i, 5.052511e-01-8.751206e-01i, 8.751206e-01+5.052511e-01i, 1.375095e+00+6.749240e-15i

```
1 # check each candidate's coefficient SEs directly:
2 dam_fit2 |> select(sarima_p4) |> report()
```

```
1 Series: current_mean
2 Model: ARIMA(4,0,0)(0,1,1)[12]
3
4 Coefficients:
5      ar1      ar2      ar3      ar4      sma1
6  1.3909 -0.4970  0.1240 -0.0413 -0.8560
7 s.e.  0.0602  0.1029  0.1022  0.0598  0.0319
8
9 sigma^2 estimated as 14.08:  log likelihood=-756.14
10 AIC=4026.81  AICc=4027.12  BIC=4048.53
```

```
1 dam_fit2 |> select(sarima_p2) |> report()
```

```
1 Series: current_mean
2 Model: ARIMA(2,0,0)(0,1,1)[12]
3
4 Coefficients:
```

```
5           ar1           ar2           sma1
6           1.3448      -0.3695      -0.8518
7 s.e.      0.0553       0.0554       0.0318
8
9 sigma^2 estimated as 14.2:  log likelihood=-757.26
10 AIC=4025.04    AICc=4025.19    BIC=4039.52
```

Model fitting

I have naively fitted MA,AR,ARMA,ARIMA,SARIMA here just to get a gauge of how they look.

Dam levels

```
1 # A mable: 6 x 2
2 # Key:      Model [6]
3 Model      Fit
4 <chr>      <model>
5 1 ar      <ARIMA(1,0,0) w/ mean>
6 2 ma      <ARIMA(0,0,1) w/ mean>
7 3 arma     <ARIMA(1,0,1) w/ mean>
8 4 arima    <ARIMA(1,1,1)>
9 5 sarima   <ARIMA(1,0,1)(1,1,1)[12]>
10 6 auto_sarima <ARIMA(3,0,1)(0,1,1)[12]>
```

Sea surface temperature

```
1 # A mable: 6 x 2
2 # Key:      Model [6]
3 Model      Fit
4 <chr>      <model>
5 1 ar      <ARIMA(1,0,0) w/ mean>
6 2 ma      <ARIMA(0,0,1) w/ mean>
7 3 arma     <ARIMA(1,0,1) w/ mean>
8 4 arima    <ARIMA(1,1,1)>
9 5 sarima   <ARIMA(1,0,1)(1,1,1)[12]>
10 6 auto_sarima <ARIMA(1,0,1)(2,1,0)[12]>
```

Project 3

- **Section & Topic:**[cite: 1]
- **Project Type:** (Implementation / Shiny App / Exploration / Simulation)[cite: 1]
- **Question / Aim:**[cite: 1]
- **Data Used:** (Include source details if real-world data)[cite: 1]
- **Methods Used:**[cite: 1]

Key Findings

[cite: 1]

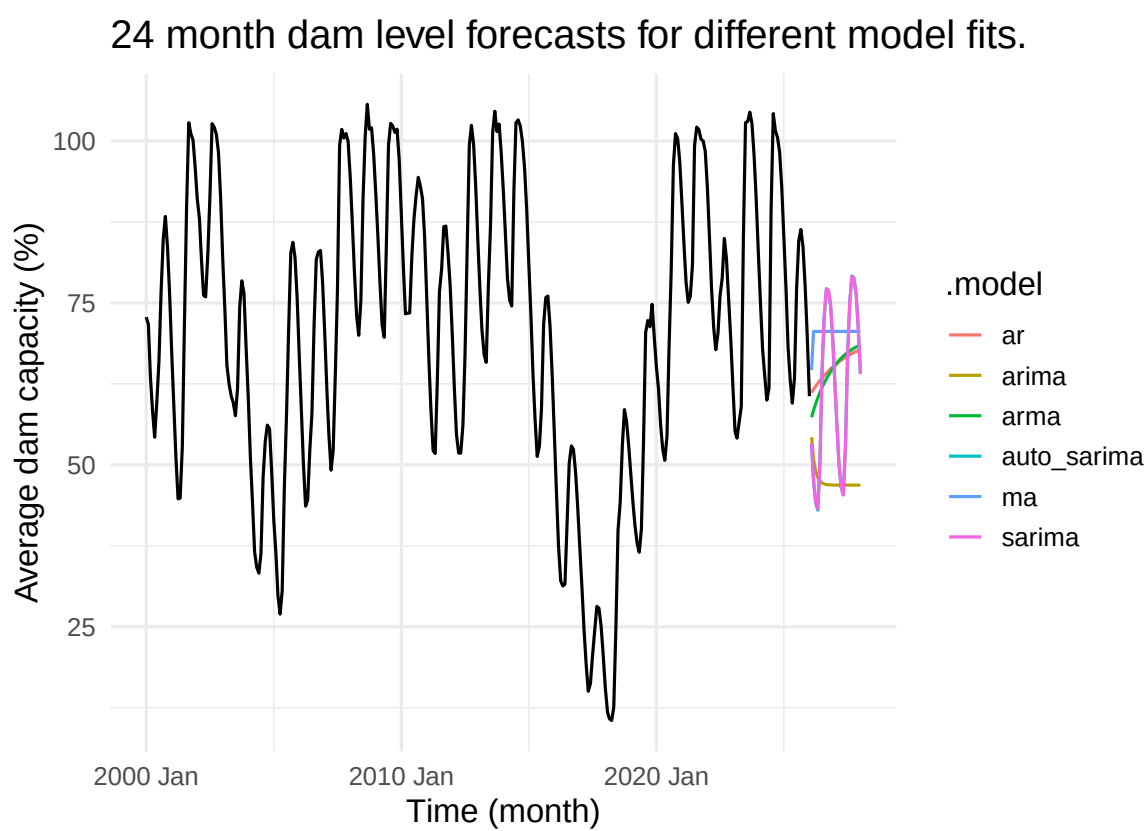


Figure 27: Naive forecasting of fitted models.

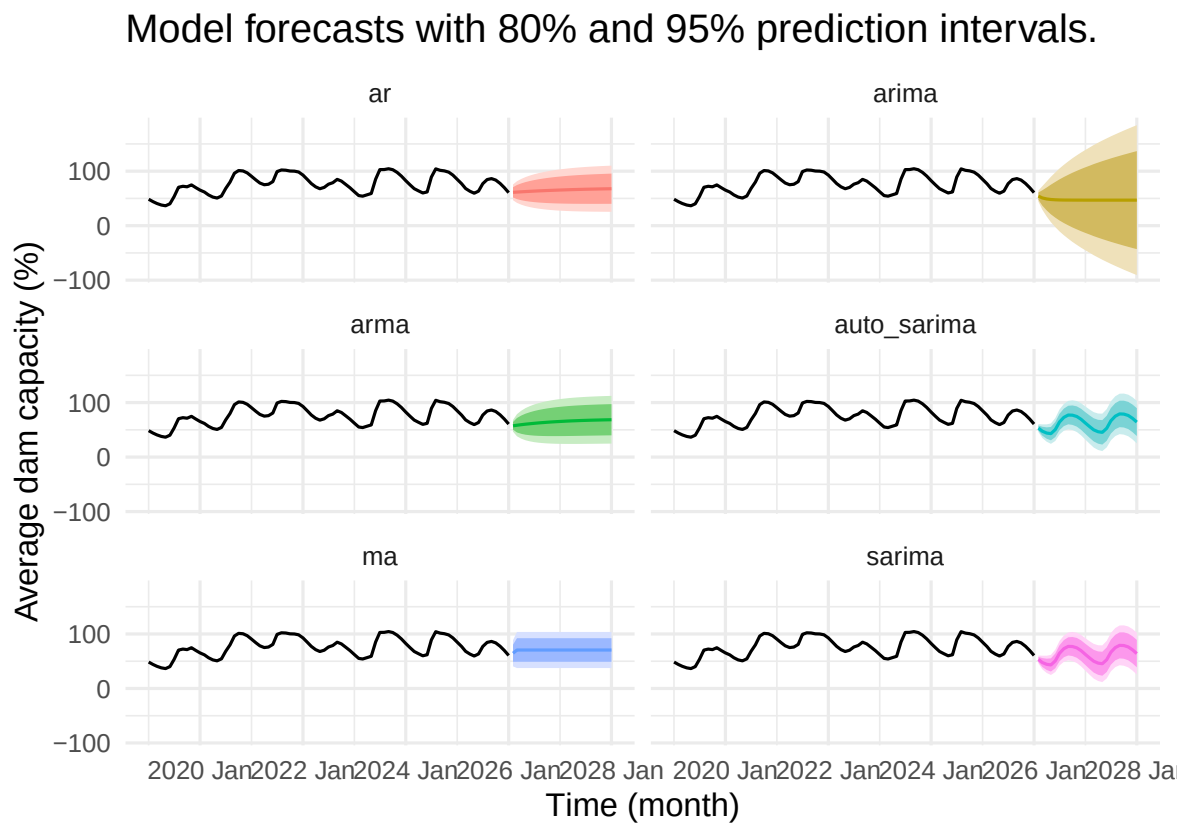


Figure 28: Naive forecasting of fitted models.

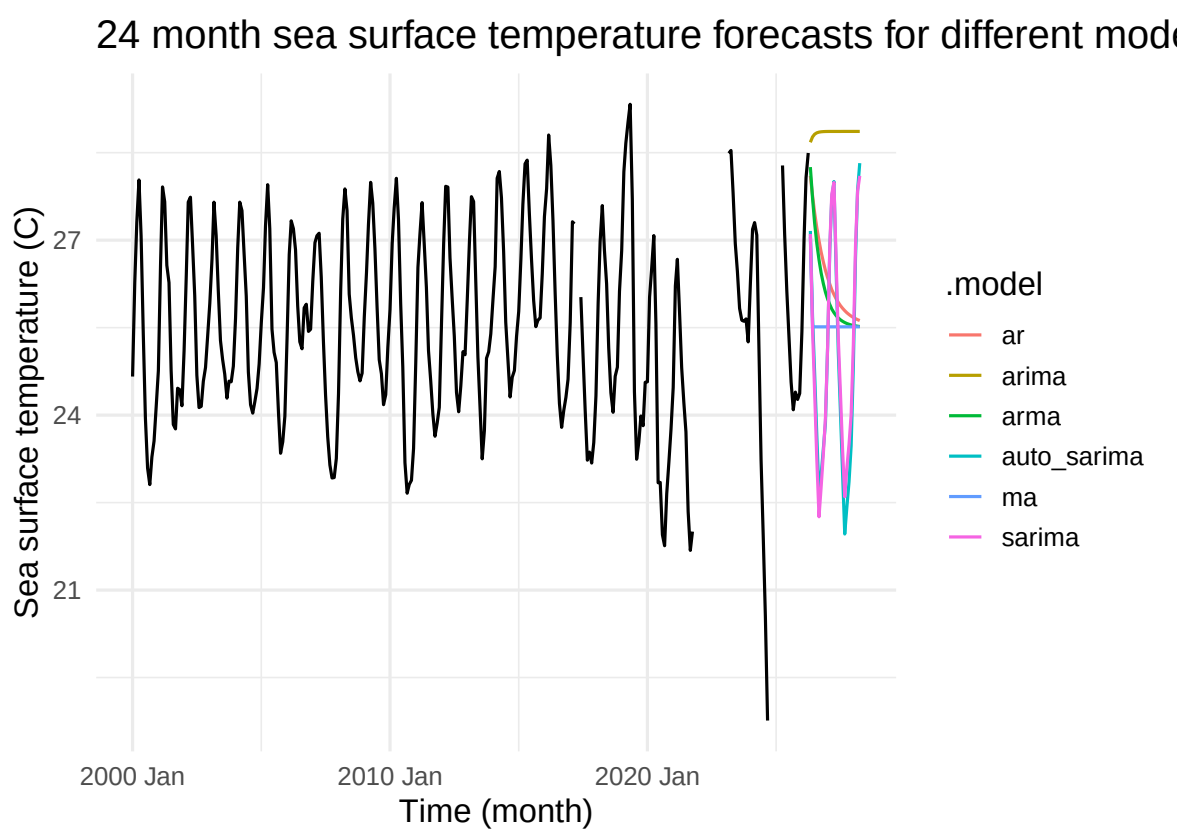


Figure 29: Naive forecasting of fitted models.

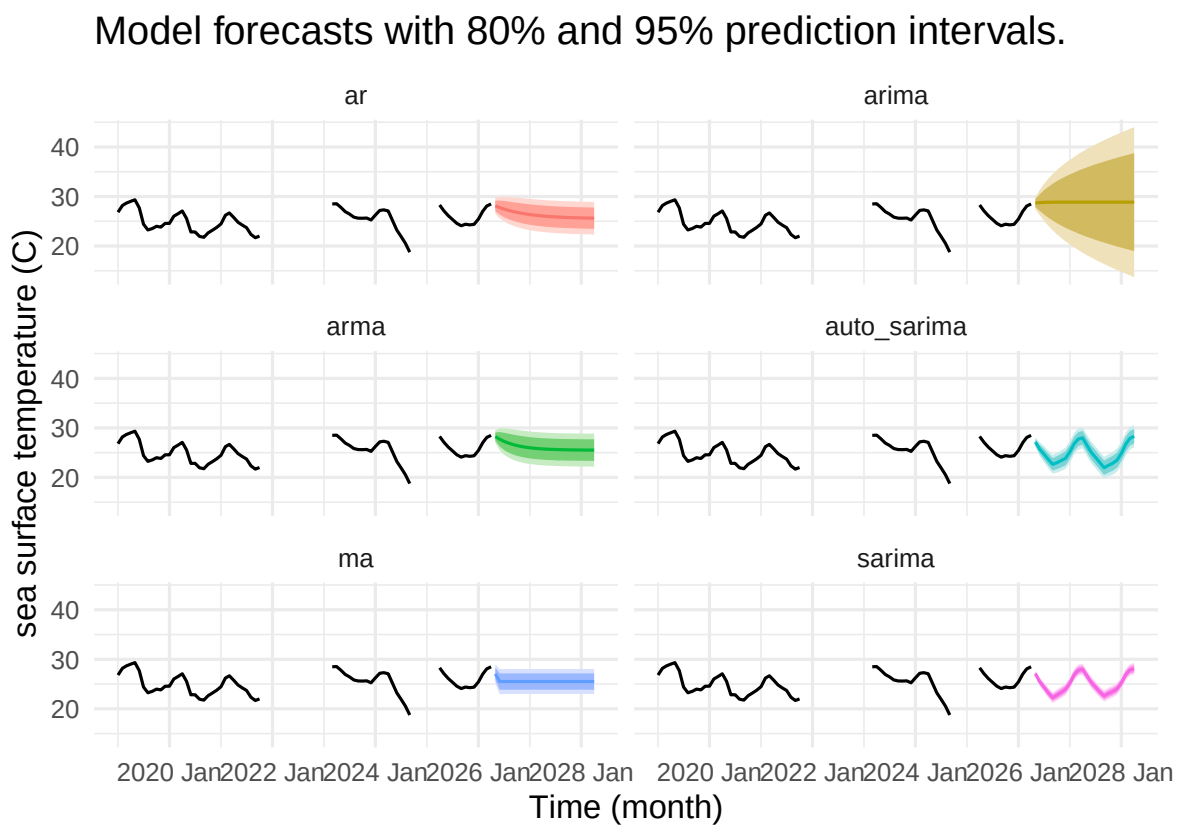


Figure 30: Naive forecasting of fitted models.

Reflections

- **What I would do differently next time:**[cite: 1]

Links

- [Detailed Analysis & Code Script](#)[cite: 1]

Project 4

- **Section & Topic:**[cite: 1]
- **Project Type:** (Implementation / Shiny App / Exploration / Simulation)[cite: 1]
- **Question / Aim:**[cite: 1]
- **Data Used:** (Include source details if real-world data)[cite: 1]
- **Methods Used:**[cite: 1]

Key Findings

[cite: 1]

Reflections

- **What I would do differently next time:**[cite: 1]

Links

- [Detailed Analysis & Code Script](#)[cite: 1]

Journal

This journal documents my continuous learning progress throughout the course[cite: 1].

- [View Chronological Entries](#)[cite: 1]

Appendix

R Code & Scripts

- `scripts/simulation-study.R`
- `scripts/shiny-app/app.R`

Data Access & Repositories

- [Raw Data Files & Data Access Instructions](#)[cite: 1]

Computational Notebooks

- Extended Analysis Reports[cite: 1]